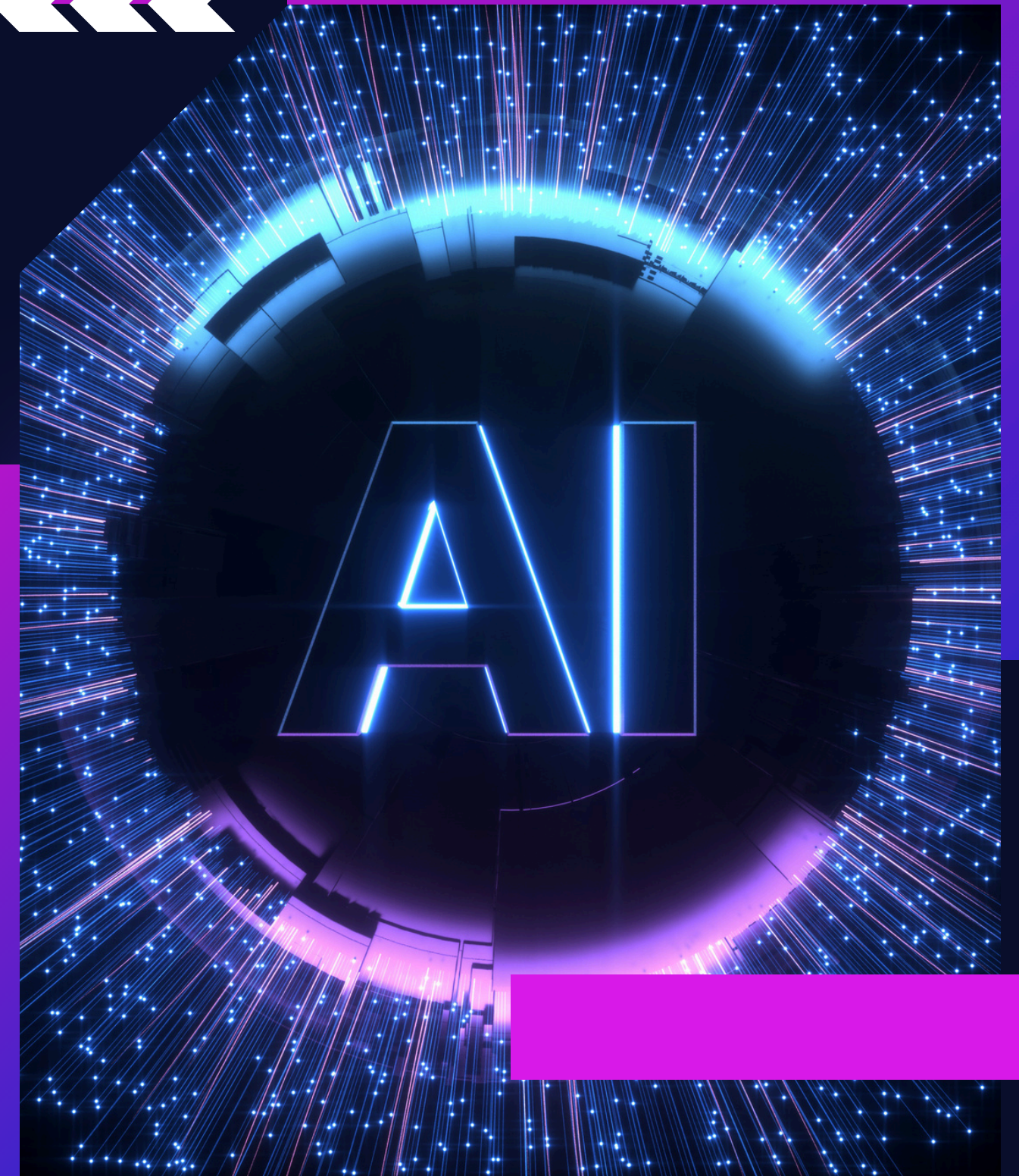


บทที่ 3 : การเรียนรู้ของเครื่อง

- 3.1 แนะนำการเรียนรู้ของเครื่อง (Introduction to ML)
- 3.2 การถดถอยเชิงเส้น (Linear Regression)
- 3.3 ลอจิสติกรีเกรสชัน (Logistic Regression: LR)
- 3.4 เค-เนียร์เรสต์เนบอร์ (K-Nearest Neighbours: KNN)
- 3.5 การตรวจสอบโมเดล (Model Validation)
- 3.6 การประเมินประสิทธิภาพโมเดล (Performance Evaluation)
- 3.7 แบบฝึกหัด (Exercise)

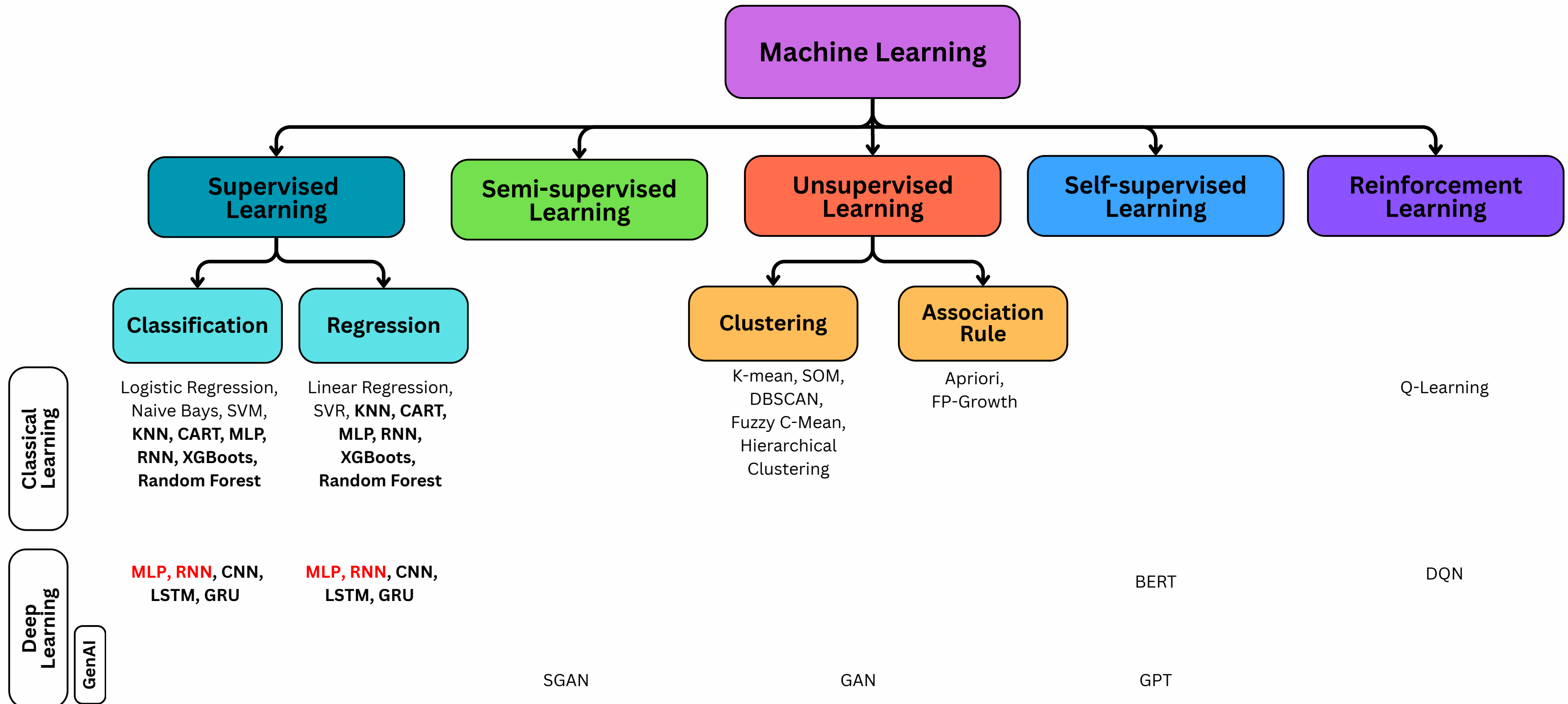


3.1 แนะนำการเรียนรู้ของเครื่อง

การเรียนรู้ของเครื่อง (Machine Learning: ML) คือ กระบวนการที่ทำให้คอมพิวเตอร์สามารถเรียนรู้จากข้อมูล และปรับปรุงการทำงานได้โดยไม่ต้องถูกเขียนโปรแกรมแบบตายตัว แบ่งออกเป็น 5 ประเภท ดังนี้

- Supervised Learning : การเรียนรู้แบบมีผู้สอน
- Semi-supervised Learning : การเรียนรู้แบบผสมระหว่างมีและไม่มีผู้สอน
- Unsupervised Learning : การเรียนรู้แบบไม่มีผู้สอน
- Self-supervised Learning : การเรียนรู้ด้วยการสอนตัวมันเอง
- Reinforcement Learning : การเรียนรู้จากสภาพแวดล้อม โดยการลงโทษและการให้รางวัล

3.1.1 ประเภทและเทคนิคการเรียนรู้ของเครื่อง



ภาพที่ 3-1 : ประเภทและเทคนิคการเรียนรู้ของเครื่อง

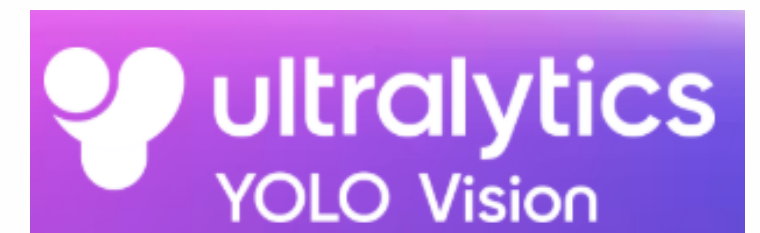
3.1.2 เครื่องมือสำหรับการเรียนรู้ของเครื่อง

- **Library:**

- Scikit-Learn
- TensorFlow (Google)
- PyTorch (Meta/Facebook)
- OpenCV
- YOLO (Ultralytics)

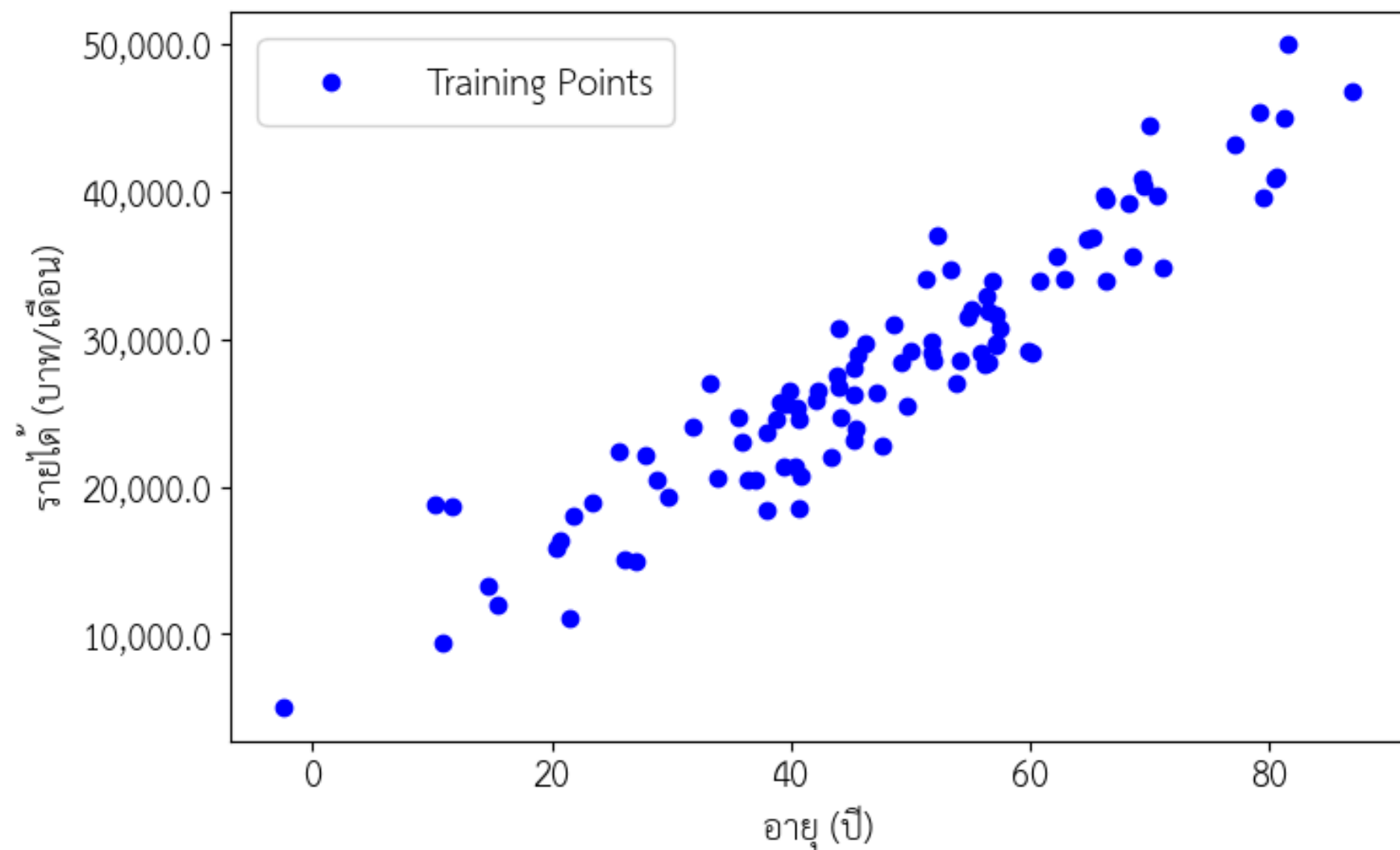
- **Programming Language:**

- Python (นิยมมากที่สุดในงาน ML)

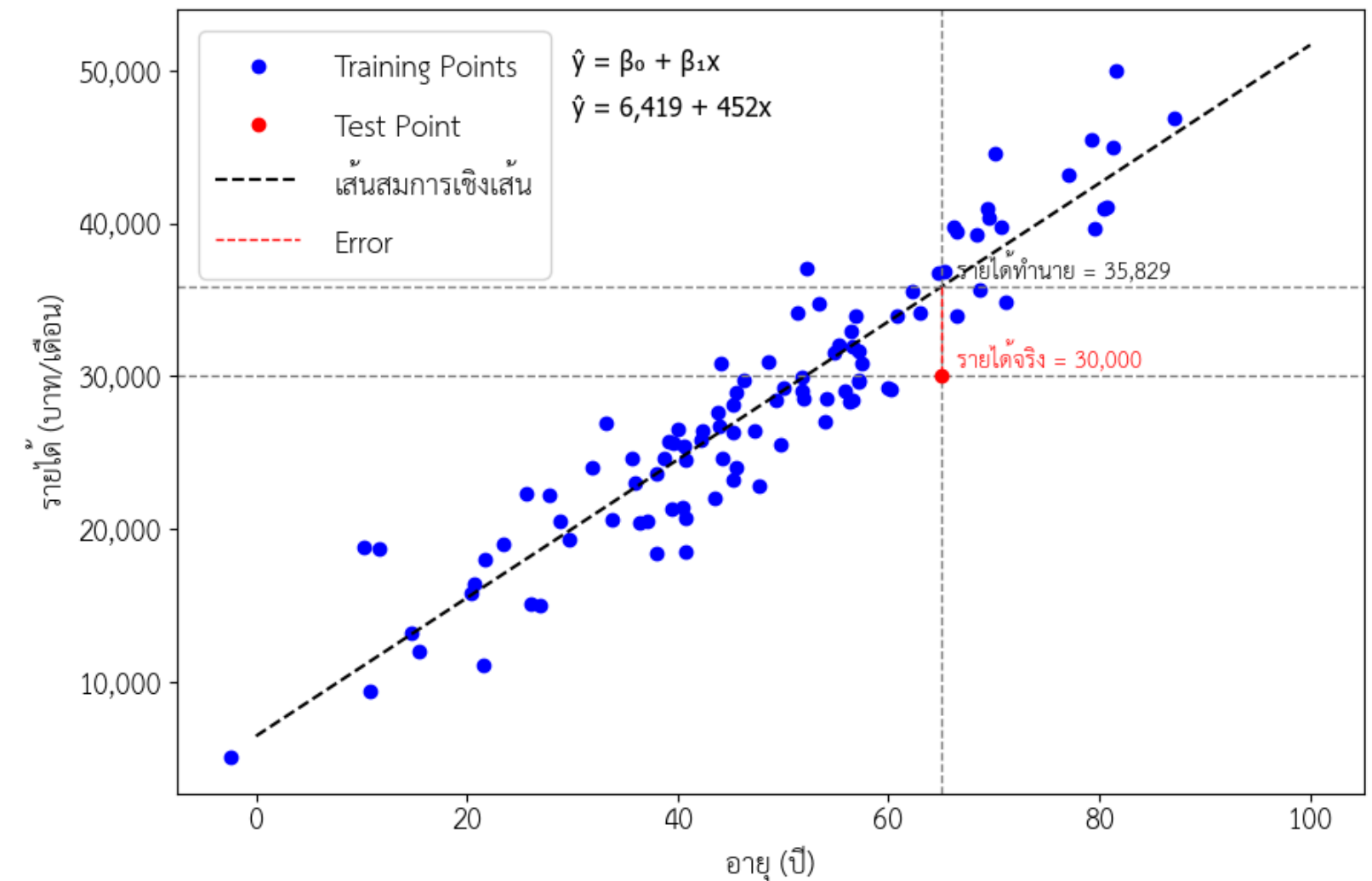


3.2 การถดถอยเชิงเส้น (Linear Regression)

- มีเป้าหมายเพื่อประมาณค่าพารามิเตอร์ β ของสมการเชิงเส้น
- เพื่อหาความสัมพันธ์ระหว่างตัวแปรต้น (X) กับ ตัวแปรตาม (Y) โดยให้เส้นตรง มีระยะห่างจากข้อมูลจริงน้อยที่สุด
- ช่วยต่อการทำความเข้าใจและการคำนวณ และเหมาะกับข้อมูลที่มีความสัมพันธ์เชิงเส้น
- Sensitive ต่อ Outliers ส่งผลให้ ค่า β เบี่ยงเบนจากค่าที่ควรจะเป็น ควรลบ outliers หรือ take log
- วิธีประมาณพารามิเตอร์ เช่น Ordinary Least Square (OLS), Lasso, Ridge , Gradient Descent



ภาพที่ 3-2ก : ชุดข้อมูลฝึก (Training Set)



ภาพที่ 3-2ข : การทดสอบ (Test)

ภาพที่ 3-2 : ตัวอย่างการประมาณการรายได้ด้วย Linear Regression

3.2.1 การคำนวณสำหรับ OLS

ขั้นที่ 1 : คำนวณซูโดอินเวอร์ส (Pseudo Inverse) ของ $\mathbf{X}$

$$\mathbf{X}^+ = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$$

เมื่อ $\mathbf{X}$ คือ เมตริกของชุดข้อมูลฝึก (ตัวแปรต้น), $\mathbf{X}^T$ คือ ทรานสโพส (Transpose) ของ $\mathbf{X}$

$\mathbf{X}^+$ คือ ซูโดอินเวอร์ส (Pseudo inverse) ของ $\mathbf{X}$

$(\mathbf{X}^T \mathbf{X})^{-1}$ คือ อินเวอร์ส ของผลคูณระหว่าง $\mathbf{X}^T$ กับ $\mathbf{X}$

ขั้นที่ 2 : คำนวณค่าสัมประสิทธิ์ (Coefficient) ของ $\mathbf{X}$

$$\boldsymbol{\beta} = \mathbf{X}^+ \mathbf{y}$$

$$\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

เมื่อ $\boldsymbol{\beta}$ คือ เวกเตอร์ของค่าสัมประสิทธิ์

$\mathbf{y}$ คือ เวกเตอร์ของข้อมูลฝึก (ตัวแปรตาม)

ขั้นที่ 3 : นำ $\boldsymbol{\beta}$ มาสร้างเป็นสมการ ได้ดังนี้

$$\hat{y} = \mathbf{x} \boldsymbol{\beta}$$

เมื่อ $\hat{y}$ คือ ผลการทำนาย, $\mathbf{x}$ คือ เวกเตอร์ของข้อมูลทดสอบ และเมื่อกระจายผลคูณจะได้

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

เมื่อ m คือ จำนวนตัวแปรต้น, β_m คือค่าสัมประสิทธิ์ตัวที่ m , x_m คือค่าของตัวแปรต้นตัวที่ m

เพื่อให้สมการมี β_0 จำเป็นต้องเพิ่มเวกเตอร์ 1 เข้าไปยังเมตริก $\mathbf{X}$ ในขั้นที่ 1

3.2.2 ตัวอย่างการคำนวณ OLS

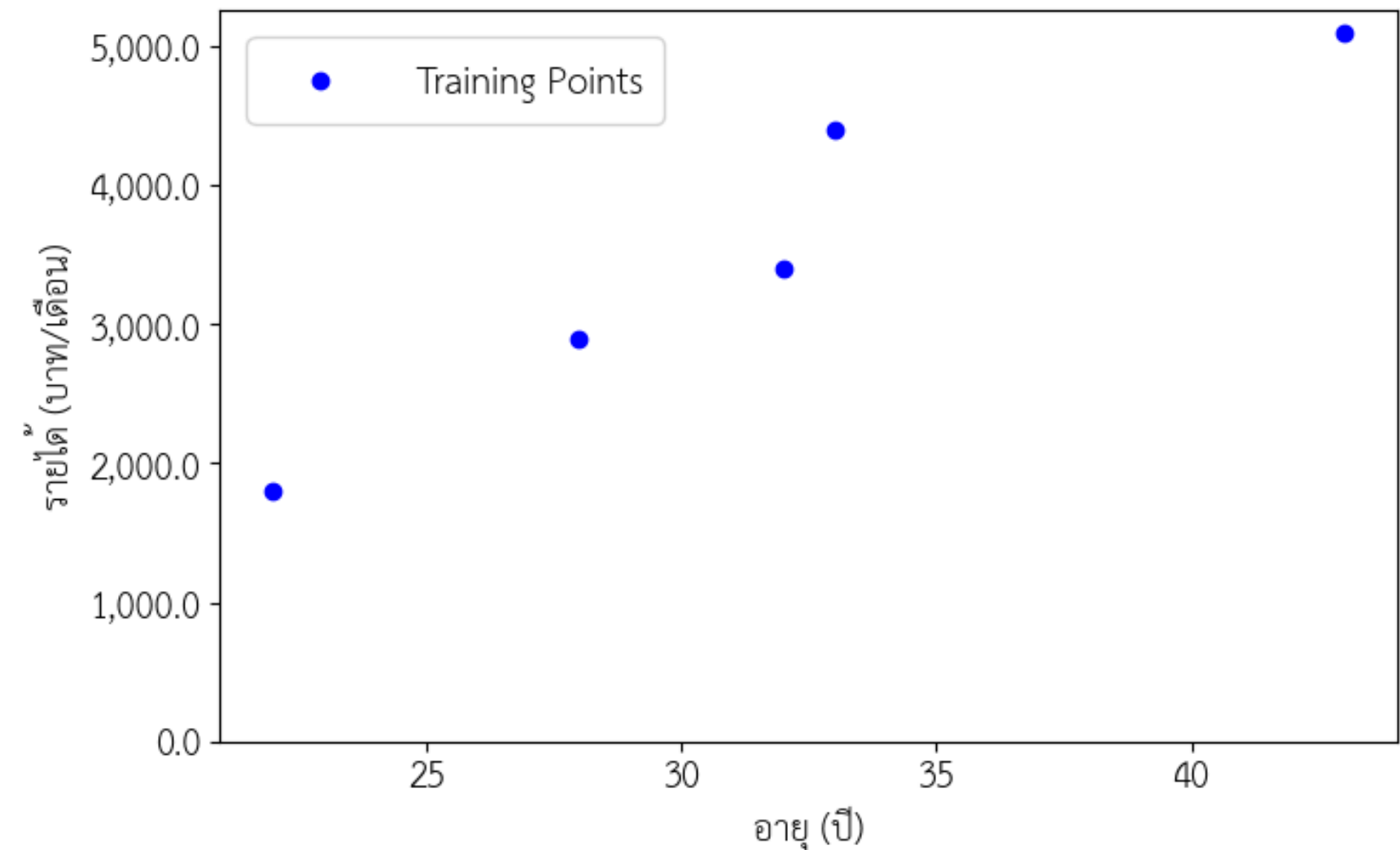
จงแสดงวิธีการคำนวณ OLS เพื่อประมาณการรายได้

ตารางที่ 3-1ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ (ปี)	รายได้ (บาท)
1	22	1800
2	28	2900
3	32	3400
4	33	4400
5	43	5100

ตารางที่ 3-1ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	รายได้ (บาท)
1	20	?
2	30	?



ภาพที่ 3-3 : ชุดข้อมูลฝึก (Training Set)

วิธีทำ

ขั้นที่ 1 : คำนวณซูโดอินเวิร์ส (Pseudo Inverse) เมื่อ $\mathbf{X}^+ = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$

กำหนดให้

$$\mathbf{X} = \begin{bmatrix} 1 & 22 \\ 1 & 28 \\ 1 & 32 \\ 1 & 33 \\ 1 & 43 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 1800 \\ 2900 \\ 3400 \\ 4400 \\ 5100 \end{bmatrix} \quad \mathbf{X}^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 22 & 28 & 32 & 33 & 43 \end{bmatrix}$$

จะได้

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 22 & 28 & 32 & 33 & 43 \end{bmatrix} \times \begin{bmatrix} 1 & 22 \\ 1 & 28 \\ 1 & 32 \\ 1 & 33 \\ 1 & 43 \end{bmatrix} = \begin{bmatrix} 5 & 158 \\ 158 & 5230 \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{1186} \begin{bmatrix} 5230 & -158 \\ -158 & 5 \end{bmatrix} = \begin{bmatrix} 4.40978 & -0.13322 \\ -0.13322 & 0.00422 \end{bmatrix}$$

$$\begin{aligned} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T &= \begin{bmatrix} 4.40978 & -0.13322 \\ -0.13322 & 0.00422 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 22 & 28 & 32 & 33 & 43 \end{bmatrix} \\ &= \begin{bmatrix} 1.47892 & 0.67959 & 0.14671 & 0.01349 & -1.3187 \\ -0.04047 & -0.01517 & 0.00168 & 0.00590 & 0.04806 \end{bmatrix} \end{aligned}$$

ขั้นที่ 2 : คำนวณค่าสัมประสิทธิ์ เมื่อ $\beta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

$$(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \begin{bmatrix} 1.47892 & 0.67959 & 0.14671 & 0.01349 & -1.3187 \\ -0.04047 & -0.01517 & 0.00168 & 0.00590 & 0.04806 \end{bmatrix} \times \begin{bmatrix} 1800 \\ 2900 \\ 3400 \\ 4400 \\ 5100 \end{bmatrix}$$
$$= \begin{bmatrix} -1534.40 \\ 159.95 \end{bmatrix}$$

ขั้นที่ 3 : นำ β มาสร้างสมการเชิงเส้น ได้ดังนี้

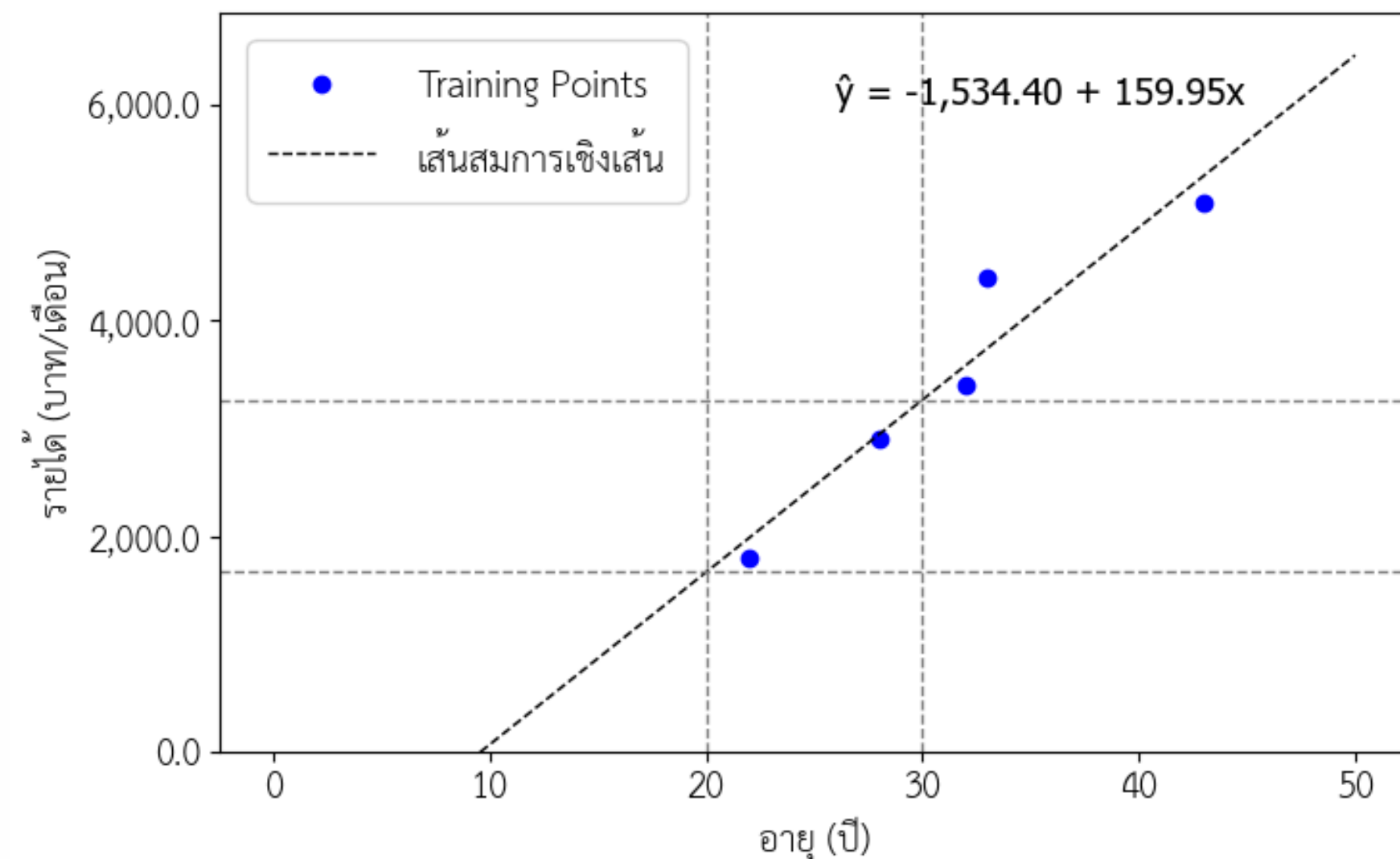
$$\hat{y} = -1,534.40 + 159.95x_1$$

เมื่อ $x_1 = 20$

$$\hat{y} = -1534.40 + 159.95 \times 20 = 1,664.6$$

เมื่อ $x_1 = 30$

$$\hat{y} = -1534.40 + 159.95 \times 30 = 3,264.1$$



ภาพที่ 3-4 : การทดสอบ (Test) หรือการประมาณรายได้

3.2.3 ตัวอย่างโค้ด OLS

- เขียนโค้ดแบบไม่ใช่ไลบรารี ML

```
1 #Import Python Library
2 import numpy as np
3 from numpy.linalg import inv
4
5 #กำหนดค่าเริ่มต้น
6 x_train = np.array([[1,22], [1,28], [1,32], [1,33], [1,43]])
7 y_train = np.array([1800, 2900, 3400, 4400, 5100])
8 x_test = np.array([20, 30])
9
10 #ขั้นที่ 1 : คำนวณซุโดอินเวอร์ส (Pseudo Inverse) เมื่อ  $X^+ = [(X^T X)]^{-1} X^T$ 
11 mat_trans = np.transpose(x_train) #  $x^T$ 
12 mat_mul = np.matmul(mat_trans, x_train) #  $X^T X$ 
13 inverse = np.linalg.inv(mat_mul) #  $[(X^T X)]^{-1}$ 
14 pseudo_inv = np.matmul(inverse, mat_trans) #  $(x^T x)^{-1} * x^T$ 
15 print(f'ขั้นที่ 1 : ซุโดอินเวอร์ส คือ {pseudo_inv}')
16
17 #ขั้นที่ 2 : คำนวณค่าสัมประสิทธิ์ เมื่อ  $\beta = [(X^T X)]^{-1} X^T y$ 
18 beta = np.matmul(pseudo_inv, y_train)
19 print(f'ขั้นที่ 2 : ค่าสัมประสิทธิ์ คือ {beta}')
20
21 #ขั้นที่ 3 : สร้างสมการเชิงเส้นเพื่อทำนาย
22 print(f'ขั้นที่ 3 : สมการเชิงเส้น คือ  $y = \{round(beta[0], 2)\} + \{round(beta[1], 2)\}x$ ')
23 for x in x_test:
24     print(f'\tเมื่ออายุ {x} ปี รายได้จากการประเมิน คือ {(beta[0] + beta[1] * x):.2f} บาท/เดือน')
```

```
ขั้นที่ 1 : ซุโดอินเวอร์ส คือ [[ 1.47892074  0.67959528  0.14671164  0.01349073 -1.31871838]
[-0.04047218 -0.01517707  0.00168634  0.00590219  0.04806071]]
ขั้นที่ 2 : ค่าสัมประสิทธิ์ คือ [-1534.40134907  159.94940978]
ขั้นที่ 3 : สมการเชิงเส้น คือ  $y = -1534.4 + 159.95x$ 
          เมื่ออายุ 20 ปี รายได้จากการประเมิน คือ 1,664.59 บาท/เดือน
          เมื่ออายุ 30 ปี รายได้จากการประเมิน คือ 3,264.08 บาท/เดือน
```

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

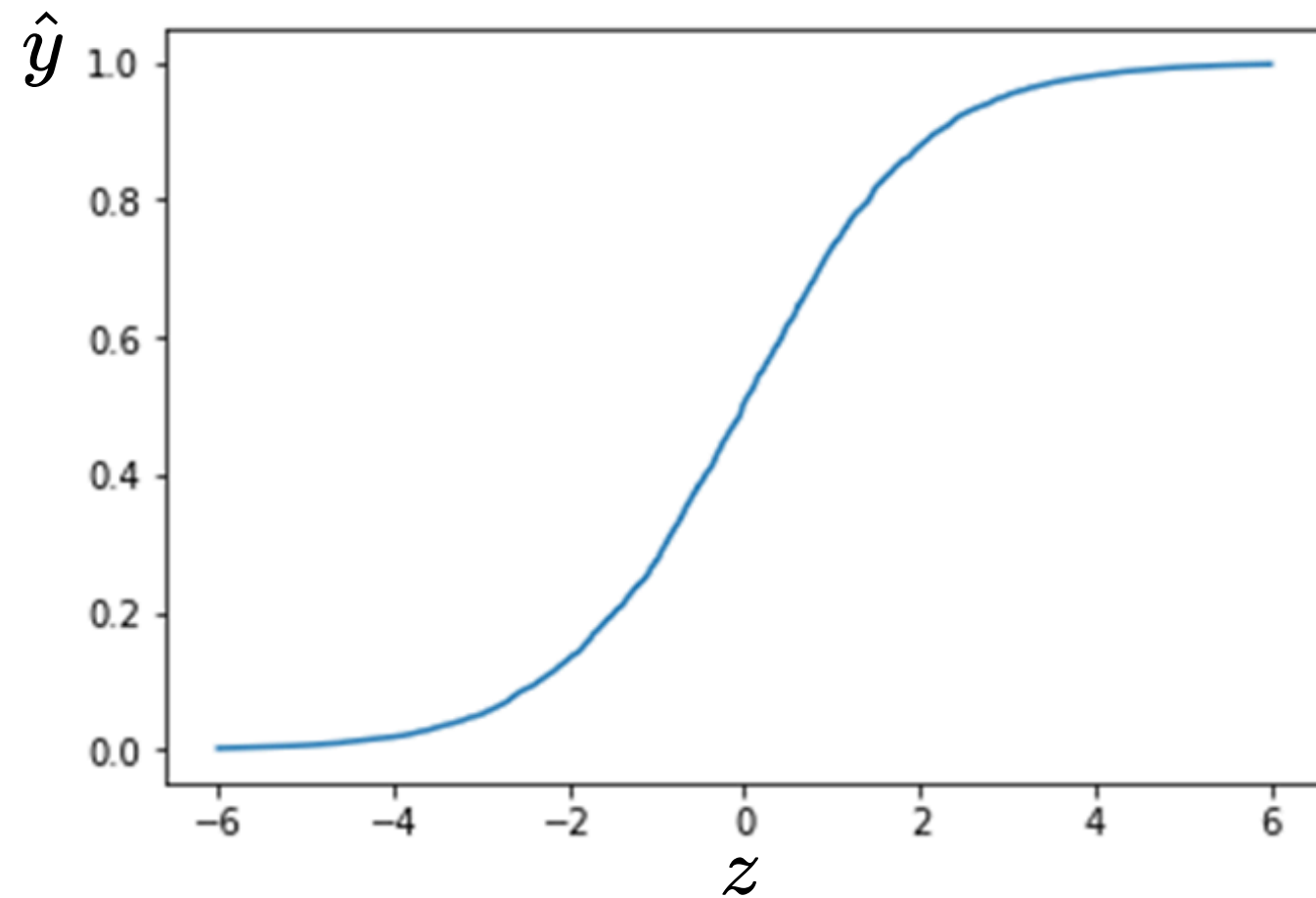
```
1 #Import Python Library
2 import numpy as np
3 from sklearn import linear_model
4
5 #Load Data
6 x_train = np.array([22, 28, 32, 33, 43]).reshape(-1, 1)
7 y_train = np.array([1800, 2900, 3400, 4400, 5100])
8 x_test = np.array([20, 30]).reshape(-1, 1)
9
10 #Build Model
11 net=linear_model.LinearRegression()
12 net.fit(x_train, y_train)
13
14 #Test Model
15 y_pred=net.predict(x_test)
16 #print(f'สมการเชิงเส้น คือ y={net.intercept_:,.2f} + {net.coef_[0]:,.2f}x')
17 for i, x in enumerate(x_test):
18     print(f'\tเมื่ออายุ {x[0]} ปี รายได้จากการประเมิน คือ {y_pred[i]:,.2f} บาท/เดือน')
```

เมื่ออายุ 20 ปี รายได้จากการประเมิน คือ 1,664.59 บาท/เดือน

เมื่ออายุ 30 ปี รายได้จากการประเมิน คือ 3,264.08 บาท/เดือน

3.3 โลจิสติกเรเกรสชัน (LR)

- อาศัยแนวคิดของความน่าจะเป็นและฟังก์ชันโลจิสติก (Sigmoid) เพื่อประมาณค่าความน่าจะเป็น ของการเกิดเหตุการณ์ของ ตัวแปรตาม จาก ตัวแปรต้น
- ค่าที่ได้จะอยู่ในช่วง 0 ถึง 1 ซึ่งแทนค่าความน่าจะเป็น (Probability)
- Sigmoid Function ถือเป็นหัวใจสำคัญที่ใช้แปลงค่าจากสมการเส้นตรง (Linear) ให้กลายเป็นเส้นโค้งรูปตัว S เพื่อจำกัดค่าให้อยู่ระหว่าง 0 กับ 1
- การตัดสินใจจะใช้ ค่า Threshold เช่น ถ้าความน่าจะเป็น > 0.5 ให้ถือว่าเป็น class 1 ถ้าต่ำกว่าให้เป็น class 0



ภาพที่ 3-5 : Logistic Sigmoid Function

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

$$P(y = 1|x) = \hat{y} = \frac{1}{1 + e^{-z}}$$

3.3.1 การคำนวณสำหรับ LR

ขั้นตอนการคำนวณ

- การเตรนโมเดล

ขั้นที่ 0 : กำหนดค่าเริ่มต้น

- การกำหนด Optimizer (Solver)
 - Gradient Descent (GD)
 - LBFGS
- การกำหนด Learning Rate (η)
 - ถ้าสูงเกินไป การเรียนรู้จะแกว่ง (divergence) มาก
 - ถ้าต่ำเกินไปจะเรียนรู้จบ (convergence) ช้า
- กำหนดค่า weight เริ่มต้น $\beta_0, \beta_1, \beta_2 \dots, \beta_m$
โดยการสุ่มค่าขนาดเล็ก
- การกำหนด จำนวน epoch
 - ถ้าข้อมูลขนาดใหญ่จำนวนรอบอาจน้อยลง
 - ถ้าข้อมูลขนาดเล็กจำนวนรอบอาจมากขึ้น

ขั้นที่ 1 : คำนวณค่า Linear Combination และ Probability

- Linear Combination

$$z_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_m x_{im} = \beta^T x_i$$

$$= \beta_0 + \sum_{j=1}^m \beta_j x_{ij} = \sum_{j=0}^m \beta_j x_{ij} \text{ เมื่อ } x_{i0} = 1$$

- Probability

$$P(y_i = 1|x_i) = \hat{y}_i = \frac{1}{1 + e^{-z_i}} \text{ เมื่อ } \hat{y}_i \text{ คือ ความน่าจะเป็นที่โมเดลทำนาย ของข้อมูลตัวที่ } i$$

ขั้นที่ 2 : คำนวณ Loss (Error)

$$L(\beta) = \frac{1}{n} \sum_{i=1}^n [-y_i \ln(\hat{y}_i) - (1 - y_i) \ln(1 - \hat{y}_i)]$$

ขั้นที่ 3 : คำนวณ Gradient โดยใช้ Chain Rule

$$\frac{\partial L}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n \frac{\partial L}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \beta_j}$$

- คำนวณแต่ละส่วน

$$(1) \quad \frac{\partial L}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i} + \frac{1 - y_i}{1 - \hat{y}_i}$$

$$(2) \quad \frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$$

(3) คูณ (1) และ (2)

$$\frac{\partial L}{\partial z_i} = \left(-\frac{y_i}{\hat{y}_i} + \frac{1 - y_i}{1 - \hat{y}_i} \right) \hat{y}_i(1 - \hat{y}_i) = \hat{y}_i - y_i$$

$$(4) \quad \frac{\partial z_i}{\partial \beta_j} = x_{ij}$$

- รวมทุกส่วนเข้าด้วยกัน

$$\frac{\partial L}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

ขั้นที่ 4 : ปรับค่า Weight

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \eta \frac{\partial L}{\partial \beta_j}$$

t คือ รอบการปรับ weight

$t = 0$ เป็นการกำหนด weight เริ่มต้น

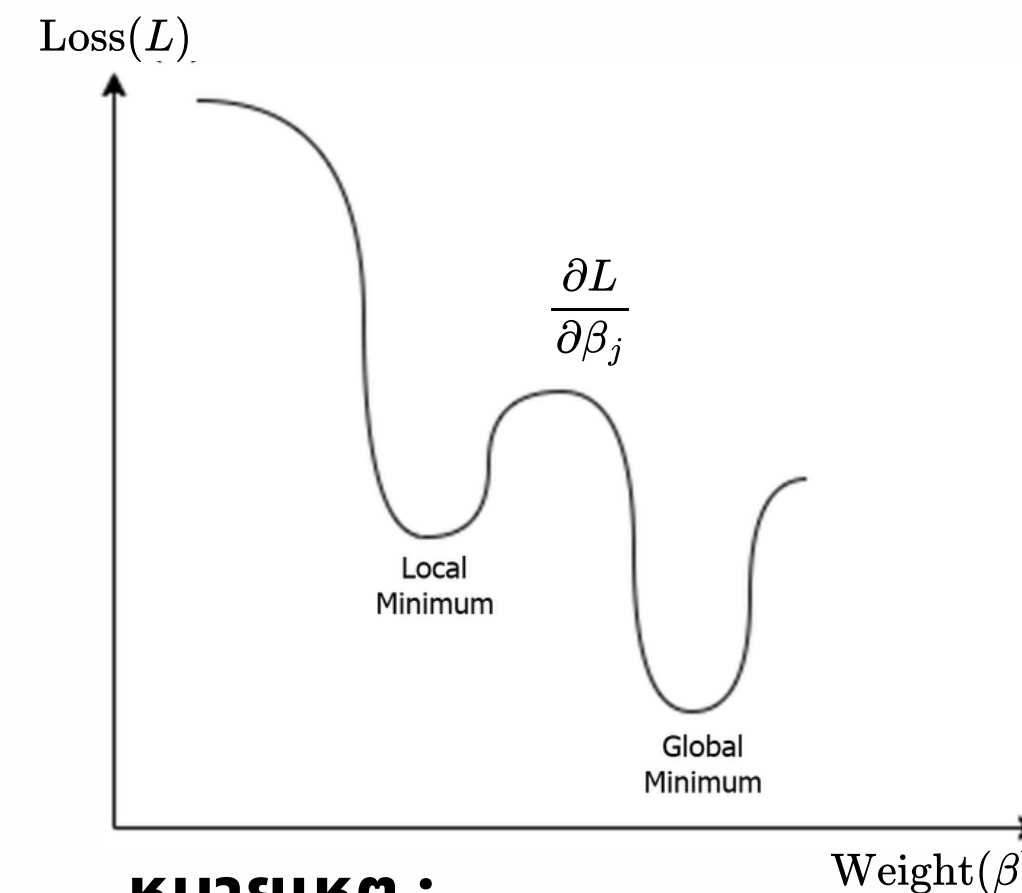
η คือ Learning Rate

ลบ gradient เพื่อลงไปหาค่า Loss ต่ำสุด

ขั้นที่ 5 : กลับไปทำขั้นตอนที่ 1-4 จนกว่าจะเข้าเงื่อนไขการหยุด

หยุดเมื่อ

- Loss ลดลงน้อยมาก หรือ
- Gradient เข้าใกล้ 0 หรือ
- ครบจำนวน Epoch ที่กำหนด



หมายเหตุ :

Loss Function = ภูเขา

Weight = ตำแหน่งบนภูเขา

Gradient = ความชัน

Gradient Descent = เดินลงเขาที่ละก้าวจนถึงจุดต่ำสุด

- **การทดสอบโมเดล**

- คำนวณ Linear Combination

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

- คำนวณ Probability

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

- ทำนายผลลัพธ์

ทำนายเป็น

class label = 1 ถ้าค่า Probability ≥ 0.5

class label = 0 ถ้าค่า Probability < 0.5

3.3.2 ตัวอย่างการคำนวณ LR

จงแสดงวิธีการคำนวณ Logistic Regression เพื่อวินิจฉัยโรคเบาหวาน โดยให้ใช้ค่า และชุดข้อมูลต่อไปนี้

- Solver = GD
- จำนวน Epoch = 2
- Learning Rate = 0.0001
- ค่าเริ่มต้นของ Weight :

$$\beta_0 = -20$$

$$\beta_1 (\text{อายุ}) = 0.05$$

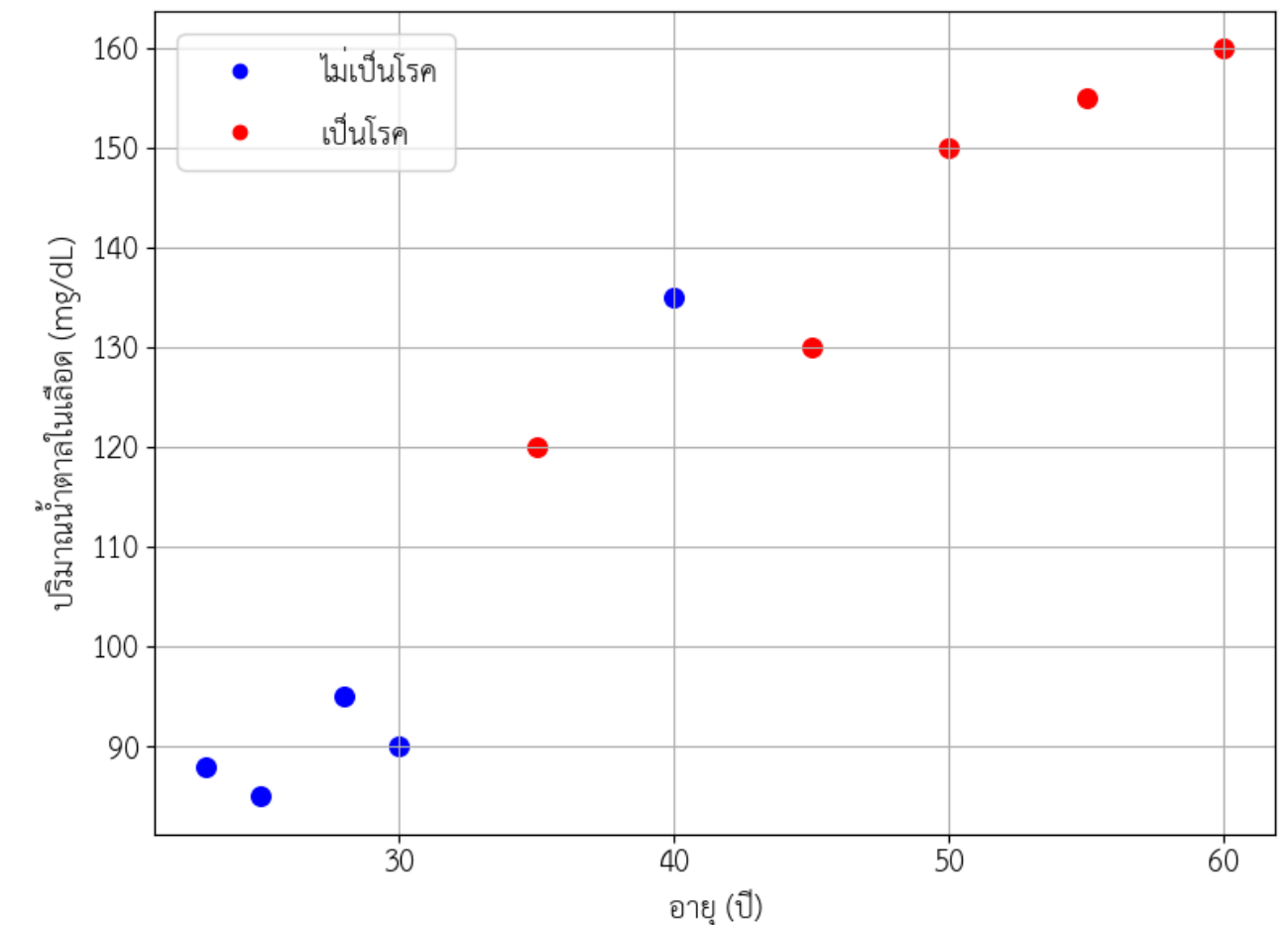
$$\beta_2 (\text{น้ำตาล}) = 0.15$$

ตารางที่ 3-2ก : ชุดข้อมูล (Training Set)

รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	25	85	ไม่เป็นโรค
2	30	90	ไม่เป็นโรค
3	45	130	เป็นโรค
4	35	120	เป็นโรค
5	50	150	เป็นโรค
6	23	88	ไม่เป็นโรค
7	60	160	เป็นโรค
8	55	155	เป็นโรค
9	40	135	ไม่เป็นโรค
10	28	95	ไม่เป็นโรค

ตารางที่ 3-2ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	38	125	?



ภาพที่ 3-6 : ชุดข้อมูลฝึก (Training Set)

วิธีทำ

- การเทรนโมเดล

ขั้นที่ 0 : กำหนดค่าเริ่มต้น

Solver = GD

จำนวน Epoch = 2

Learning Rate = 0.0001

Weight : $\beta_0 = -20$

β_1 (อายุ) = 0.05

β_2 (น้ำตาล) = 0.15

$x_1 =$ อายุ

$x_2 =$ ปริมาณน้ำตาลในเลือด

$y \in \{0, 1\}$ โดยที่ 0 = ไม่เป็นโรค, 1 = เป็นโรค

รอบที่ 1

ขั้นที่ 1+2 : คำนวณค่า Linear Combination, Probability, และ Loss

$$z_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$$

$$\hat{y}_i = \frac{1}{1 + e^{-z_i}}$$

$$L_i = -y_i \ln(\hat{y}_i) - (1 - y_i) \ln(1 - \hat{y}_i)$$

รายการ (i)	อายุ (x _{i1})	ปริมาณน้ำตาล (x _{i2})	การเป็นโรค (y _i)	linear combination (z _i)	probability (y _i ^)	loss (L _i)
1	25	85	0	-20 + 0.05x25 + 0.15x85 = -6	1 / 1 + e^(-6) = 0.0025	-0ln(0.0025) - (1-0)ln(1-0.0025) = 0.0025
2	30	90	0	-20 + 0.05x30 + 0.15x90 = -5	1 / 1 + e^(-5) = 0.0067	-0ln(0.0067) - (1-0)ln(1-0.0067) = 0.0067
3	45	130	1	-20 + 0.05x45 + 0.15x130 = 1.75	1 / 1 + e^(-1.75) = 0.852	-1ln(0.852) - (1-1)ln(1-0.852) = 0.1602
4	35	120	1	-20 + 0.05x35 + 0.15x120 = -0.25	1 / 1 + e^(-0.25) = 0.4378	-1ln(0.4378) - (1-1)ln(1-0.4378) = 0.826
5	50	150	1	-20 + 0.05x50 + 0.15x150 = 5	1 / 1 + e^(-5) = 0.9933	-1ln(0.9933) - (1-1)ln(1-0.9933) = 0.0067
6	23	88	0	-20 + 0.05x23 + 0.15x88 = -5.65	1 / 1 + e^(-5.65) = 0.0035	-0ln(0.0035) - (1-0)ln(1-0.0035) = 0.0035
7	60	160	1	-20 + 0.05x60 + 0.15x160 = 7	1 / 1 + e^(-7) = 0.9991	-1ln(0.9991) - (1-1)ln(1-0.9991) = 0.0009
8	55	155	1	-20 + 0.05x55 + 0.15x155 = 6	1 / 1 + e^(-6) = 0.9975	-1ln(0.9975) - (1-1)ln(1-0.9975) = 0.0025
9	40	135	0	-20 + 0.05x40 + 0.15x135 = 2.25	1 / 1 + e^(-2.25) = 0.9047	-0ln(0.9047) - (1-0)ln(1-0.9047) = 2.3507
10	28	95	0	-20 + 0.05x28 + 0.15x95 = -4.35	1 / 1 + e^(-4.35) = 0.0127	-0ln(0.0127) - (1-0)ln(1-0.0127) = 0.0128
						SUM = 3.3725
						$L(\beta) = \frac{1}{n} \sum_{i=1}^n L_i$ L= 1/10 (3.3725) = 0.3373

ขั้นที่ 3 : คำนวณ Gradient

รายการ (i)	อายุ (x _{i1})	ปริมาณน้ำตาล (x _{i2})	การเป็นโรค (y _i)	$\hat{y}_i$	$(\hat{y}_i - y_i)x_{i0}$	$(\hat{y}_i - y_i)x_{i1}$	$(\hat{y}_i - y_i)x_{i2}$
1	25	85	0	0.0025	(0.0025-0)*1=0.0025	(0.0025-0)*25=0.0625	(0.0025-0)*85=0.2125
2	30	90	0	0.0067	(0.0067-0)*1=0.0067	(0.0067-0)*30=0.201	(0.0067-0)*90=0.603
3	45	130	1	0.852	(0.852-1)*1=-0.148	(0.852-1)*45=-6.66	(0.852-1)*130=-19.24
4	35	120	1	0.4378	(0.4378-1)*1=-0.5622	(0.4378-1)*35=-19.677	(0.4378-1)*120=-67.464
5	50	150	1	0.9933	(0.9933-1)*1=-0.0067	(0.9933-1)*50=-0.335	(0.9933-1)*150=-1.005
6	23	88	0	0.0035	(0.0035-0)*1=0.0035	(0.0035-0)*23=0.0805	(0.0035-0)*88=0.308
7	60	160	1	0.9991	(0.9991-1)*1=-0.0009	(0.9991-1)*60=-0.054	(0.9991-1)*160=-0.144
8	55	155	1	0.9975	(0.9975-1)*1=-0.0025	(0.9975-1)*55=-0.1375	(0.9975-1)*155=-0.3875
9	40	135	0	0.9047	(0.9047-0)*1=0.9047	(0.9047-0)*40=36.188	(0.9047-0)*135=122.1345
10	28	95	0	0.0127	(0.0127-0)*1=0.0127	(0.0127-0)*28=0.3556	(0.0127-0)*95=1.2065
					SUM = 0.2098	SUM = 10.0241	SUM = 36.224
$\frac{\partial L}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)x_{ij}$					$\partial L / \partial B_0 = 1/10(0.2098) = 0.021$	$\partial L / \partial B_1 = 1/10(10.0241) = 1.0024$	$\partial L / \partial B_2 = 1/10(36.224) = 3.6224$

ขั้นที่ 4 : ปรับค่า Weight

จาก $\beta_j^{(t+1)} = \beta_j^{(t)} - \eta \frac{\partial L}{\partial \beta_j}$

$$\beta_0^{(0)} = -20 \quad \beta_1^{(0)} = 0.05 \quad \beta_2^{(0)} = 0.15 \quad \eta = 0.0001$$

$$\frac{\partial L}{\partial \beta_0} = 0.021 \quad \frac{\partial L}{\partial \beta_1} = 1.0024 \quad \frac{\partial L}{\partial \beta_2} = 3.6224$$

จะได้

$$\beta_0^{(1)} = \beta_0^{(0)} - \eta \frac{\partial L}{\partial \beta_0} = -20 - 0.0001(0.021) = -20.0000$$

$$\beta_1^{(1)} = \beta_1^{(0)} - \eta \frac{\partial L}{\partial \beta_1} = 0.05 - 0.0001(1.0024) = 0.0499$$

$$\beta_2^{(1)} = \beta_2^{(0)} - \eta \frac{\partial L}{\partial \beta_2} = 0.15 - 0.0001(3.6224) = 0.1496$$

ขั้นที่ 5 : กลับไปทำขั้นตอนที่ 1-4 จนกว่าจะเข้าเงื่อนไขการหยุด

รอบที่ 2

ขั้นที่ 1+2 : คำนวณค่า Linear Combination, Probability, และ Loss

$$z_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$$

$$\hat{y}_i = \frac{1}{1 + e^{-z_i}}$$

$$L_i = -y_i \ln(\hat{y}_i) - (1 - y_i) \ln(1 - \hat{y}_i)$$

รายการ (i)	อายุ (x _{i1})	ปริมาณน้ำตาล (x _{i2})	การเป็นโรค (y _i)	linear combination (z _i)	probability (y _i [^])	loss (L _i)
1	25	85	0	-20 + 0.0499x25 + 0.1496x85 = -6.0365	1 / 1 + e ^{^-(-6.0365)} = 0.0024	-0ln(0.0024) - (1-0)ln(1-0.0024) = 0.0024
2	30	90	0	-20 + 0.0499x30 + 0.1496x90 = -5.039	1 / 1 + e ^{^-(-5.039)} = 0.0064	-0ln(0.0064) - (1-0)ln(1-0.0064) = 0.0064
3	45	130	1	-20 + 0.0499x45 + 0.1496x130 = 1.6935	1 / 1 + e ^{^-(-1.6935)} = 0.8447	-1ln(0.8447) - (1-1)ln(1-0.8447) = 0.1688
4	35	120	1	-20 + 0.0499x35 + 0.1496x120 = -0.3015	1 / 1 + e ^{^-(-0.3015)} = 0.4252	-1ln(0.4252) - (1-1)ln(1-0.4252) = 0.8552
5	50	150	1	-20 + 0.0499x50 + 0.1496x150 = 4.935	1 / 1 + e ^{^-(-4.935)} = 0.9929	-1ln(0.9929) - (1-1)ln(1-0.9929) = 0.0071
6	23	88	0	-20 + 0.0499x23 + 0.1496x88 = -5.6875	1 / 1 + e ^{^-(-5.6875)} = 0.0034	-0ln(0.0034) - (1-0)ln(1-0.0034) = 0.0034
7	60	160	1	-20 + 0.0499x60 + 0.1496x160 = 6.93	1 / 1 + e ^{^-(-6.93)} = 0.999	-1ln(0.999) - (1-1)ln(1-0.999) = 0.001
8	55	155	1	-20 + 0.0499x55 + 0.1496x155 = 5.9325	1 / 1 + e ^{^-(-5.9325)} = 0.9974	-1ln(0.9974) - (1-1)ln(1-0.9974) = 0.0026
9	40	135	0	-20 + 0.0499x40 + 0.1496x135 = 2.192	1 / 1 + e ^{^-(-2.192)} = 0.8995	-0ln(0.8995) - (1-0)ln(1-0.8995) = 2.2976
10	28	95	0	-20 + 0.0499x28 + 0.1496x95 = -4.3908	1 / 1 + e ^{^-(-4.3908)} = 0.0122	-0ln(0.0122) - (1-0)ln(1-0.0122) = 0.0123
						SUM = 3.3568
						$L(\beta) = \frac{1}{n} \sum_{i=1}^n L_i$ L= 1/10 (3.3568) = 0.3357

ขั้นที่ 3 : คำนวณ Gradient

รายการ (i)	อายุ (x _{i1})	ปริมาณน้ำตาล (x _{i2})	การเป็นโรค (y _i)	$\hat{y}_i$	$(\hat{y}_i - y_i)x_{i0}$	$(\hat{y}_i - y_i)x_{i1}$	$(\hat{y}_i - y_i)x_{i2}$
1	25	85	0	0.0024	(0.0024-0)*1=0.0024	(0.0024-0)*25=0.06	(0.0024-0)*85=0.204
2	30	90	0	0.0064	(0.0064-0)*1=0.0064	(0.0064-0)*30=0.192	(0.0064-0)*90=0.576
3	45	130	1	0.8447	(0.8447-1)*1=-0.1553	(0.8447-1)*45=-6.9885	(0.8447-1)*130=-20.189
4	35	120	1	0.4252	(0.4252-1)*1=-0.5748	(0.4252-1)*35=-20.118	(0.4252-1)*120=-68.976
5	50	150	1	0.9929	(0.9929-1)*1=-0.0071	(0.9929-1)*50=-0.355	(0.9929-1)*150=-1.065
6	23	88	0	0.0034	(0.0034-0)*1=0.0034	(0.0034-0)*23=0.0782	(0.0034-0)*88=0.2992
7	60	160	1	0.999	(0.999-1)*1=-0.001	(0.999-1)*60=-0.06	(0.999-1)*160=-0.16
8	55	155	1	0.9974	(0.9974-1)*1=-0.0026	(0.9974-1)*55=-0.143	(0.9974-1)*155=-0.403
9	40	135	0	0.8995	(0.8995-0)*1=0.8995	(0.8995-0)*40=35.98	(0.8995-0)*135=121.4325
10	28	95	0	0.0122	(0.0122-0)*1=0.0122	(0.0122-0)*28=0.3416	(0.0122-0)*95=1.159
					SUM = 0.1831	SUM = 8.9873	SUM = 32.8777
$\frac{\partial L}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)x_{ij}$					$\partial L / \partial B_0 = 1/10(0.1831) = 0.0183$	$\partial L / \partial B_1 = 1/10(8.9873) = 0.8987$	$\partial L / \partial B_2 = 1/10(32.8777) = 3.2878$

ขั้นที่ 4 : ปรับค่า Weight

จาก $\beta_j^{(t+1)} = \beta_j^{(t)} - \eta \frac{\partial L}{\partial \beta_j}$

$$\beta_0^{(1)} = -20 \quad \beta_1^{(1)} = 0.0499 \quad \beta_2^{(1)} = 0.1496 \quad \eta = 0.0001$$

$$\frac{\partial L}{\partial \beta_0} = 0.0183 \quad \frac{\partial L}{\partial \beta_1} = 0.8987 \quad \frac{\partial L}{\partial \beta_2} = 3.2878$$

จะได้

$$\beta_0^{(2)} = \beta_0^{(1)} - \eta \frac{\partial L}{\partial \beta_0} = -20 - 0.0001(0.0183) = -20.0000$$

$$\beta_1^{(2)} = \beta_1^{(1)} - \eta \frac{\partial L}{\partial \beta_1} = 0.0499 - 0.0001(0.8987) = 0.0498$$

$$\beta_2^{(2)} = \beta_2^{(1)} - \eta \frac{\partial L}{\partial \beta_2} = 0.1496 - 0.0001(3.2878) = 0.1493$$

ขั้นที่ 5 : หยุดการเทรนโมเดล เนื่องจากครบจำนวนรอบ (Epoch)

นำ Weight ที่คำนวณได้ในรอบสุดท้ายไปใช้ในการทดสอบ (ใช้งาน) โมเดล

- การทดสอบโมเดล

จาก

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 \quad \beta_0 = -20.0000 \quad \beta_1 = 0.0498 \quad \beta_2 = 0.1493$$

$$x_1 = 38 \quad x_2 = 125$$

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

จะได้

$$z = -20 + 0.0498(38) + 0.1493(125) = 0.5549$$

$$\hat{y} = \frac{1}{1 + e^{-0.5549}} = 0.6353$$

ตอบ ค่า Probability = 0.6353 ซึ่ง ≥ 0.5 ทำให้ class label = 1
สรุปได้ว่า ผู้ที่เข้ารับการตรวจ **เป็นโรคเบาหวาน**

3.3.3 ตัวอย่างโค้ด LR

- เขียนโค้ดแบบไม่ใช้ไลบรารี ML

```
1  #Import Python library
2  import numpy as np
3
4  #Load Data
5  df = pd.read_csv('data/diabetes.csv')
6  x = df[['age', 'blood_glucose']].to_numpy()
7  y = df['diabetes'].map({'ไม่เป็นโรค': 0, 'เป็นโรค': 1})
8  x_test = np.array([38, 125])
9
10 # Parameters
11 n = len(y)
12 lr = 0.0001
13 epochs = 2
14
15 b0 = -20.0
16 b1 = 0.05
17 b2 = 0.15
18
19 # Sigmoid Function
20 def sigmoid(z):
21     return 1 / (1 + np.exp(-z))
22
```

```

23 # การเทรนโมเดล
24 for epoch in range(epochs):
25     # 1. คำนวณค่า Linear Combination และ Probability
26     z = (b0 + b1 * x[:, 0] + b2 * x[:, 1]).round(4)
27     y_hat = sigmoid(z).round(4)
28
29     # 2. คำนวณ Loss (Error)
30     loss = np.mean(-y * np.log(y_hat) - (1 - y) * np.log(1 - y_hat))
31
32
33     # 3. คำนวณ Gradients
34     db0 = np.mean(y_hat - y)
35     db1 = np.mean((y_hat - y) * x[:, 0])
36     db2 = np.mean((y_hat - y) * x[:, 1])
37
38     # 4. ปรับค่า Weight
39     b0 -= (lr * db0).round(4)
40     b1 -= (lr * db1).round(4)
41     b2 -= (lr * db2).round(4)
42
43     print(f"Epoch {epoch+1}")
44     print(f"Loss = {loss:.4f}")
45     print(f"b0 = {b0:.4f}, b1 = {b1:.4f}, b2 = {b2:.4f}\n")
46

```

```
47 # การทดสอบโมเดล
48 z_test = b0 + b1 * x_test[0] + b2 * x_test[1]
49 prob = sigmoid(z_test)
50 y_pred = 1 if prob >= 0.5 else 0
51
52 print("=== Test Result ===")
53 print(f"Probability    {prob:.4f}")
54 print(f"Predicted Class = {y_pred}")
```

Epoch 1

Loss = 0.3373

b0 = -20.0000, b1 = 0.0499, b2 = 0.1496

Epoch 2

Loss = 0.3357

b0 = -20.0000, b1 = 0.0498, b2 = 0.1493

=== Test Result ===

Probability = 0.6353

Predicted Class = 1

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

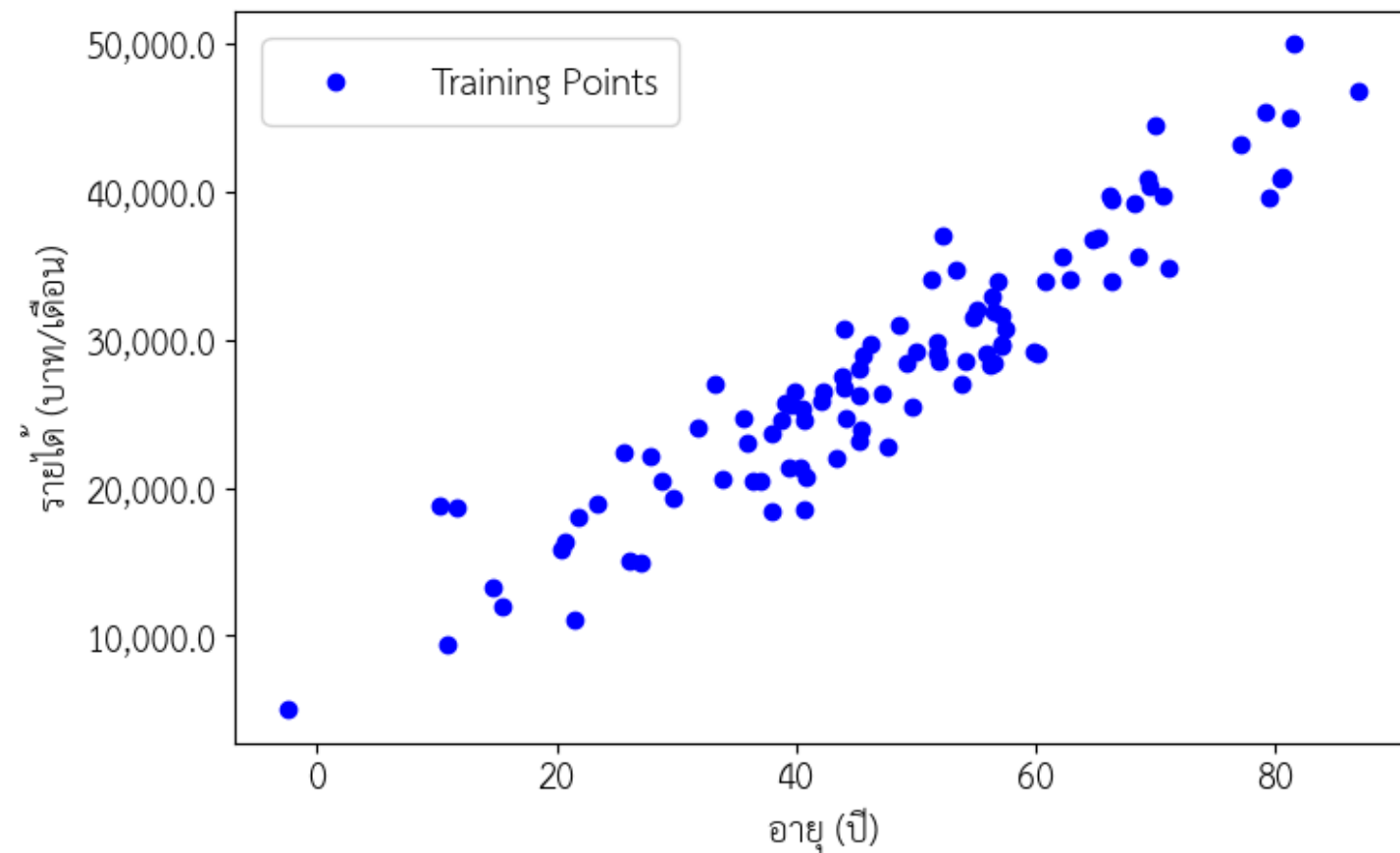
```
1 # Import Python library
2 import pandas as pd
3 from sklearn.linear_model import LogisticRegression
4
5 # Load Data
6 x = pd.read_csv('data/diabetes.csv')
7 X_train = x[['age', 'blood_glucose']].to_numpy()
8 y_train = x['diabetes'].map({'ไม่เป็นโรค': 0, 'เป็นโรค': 1})
9 X_test = [[38, 125]]
10
11 # Build and Train Model
12 model = LogisticRegression()
13 model.fit(X_train, y_train)
14
15 # Test Model
16 y_prob = model.predict_proba(X_test)
17 y_pred = model.predict(X_test)
18
19 print("=== Test Result ===")
20 print(f"Probability = {y_prob[0][1]:.4f}")
21 print(f"Predicted Class = {y_pred[0]}")
```

Probability = 0.5030

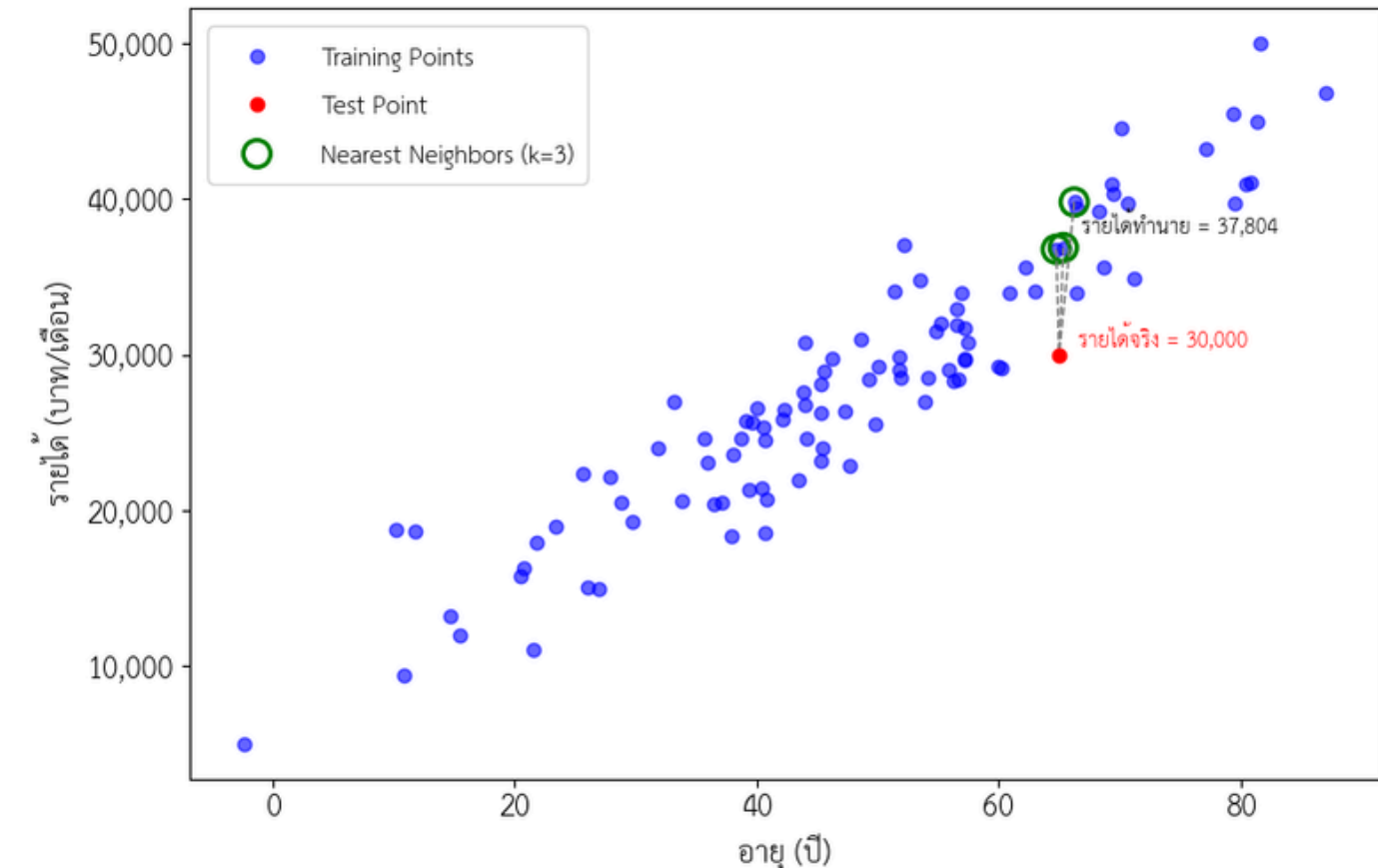
Predicted Class = 1

3.4 เค-เนียร์เรสตันเนเบอร์ (KNN)

- อาศัยหลักการวัดระยะทาง ซึ่งนิยมวัดระยะทางโดยใช้วิธี Euclidian Distance
- ง่ายต่อการทำความเข้าใจและการคำนวณ
- Sensitive ต่อ Outliers ทำให้ผลการทำนายผิดเพี้ยน ควรลบ outliers ก่อนหรือทำนายโดย weight ตามระยะทาง
- ข้อมูลที่มีช่วงต่างกันมาก เช่น อายุ [0-100] กับเงินเดือน [1 พัน, 1 แสน] จะทำให้ข้อมูลที่มีช่วงกว้างกว่า มีอิทธิพลต่อการคำนวณระยะทางมากกว่า ควรทำการ Normalize/Standardize ข้อมูล เช่น ทำให้ข้อมูลอยู่ในช่วง [0, 1]
- หากข้อมูลมีขนาดใหญ่ จะทำให้การทำนายใช้เวลานานมากเพราะต้องคำนวณทุก Training Points



ภาพที่ 3-6ก : ชุดข้อมูลฝึก (Training Set)



ภาพที่ 3-6ข : การทดสอบ (Test)

ภาพที่ 3-7 : ตัวอย่างการประมาณการรายได้ด้วย KNN

3.4.1 การคำนวณสำหรับ KNN

ขั้นที่ 1 : คำนวณระยะทาง (ความแตกต่าง) ระหว่าง test point x กับ ทั้งหมดของ training point x^n

$$d^n = d(x, x^n), \quad n = 1, \dots, N \quad \text{เมื่อ } d(x, x^n) \text{ คือ ระยะทางระหว่าง test point } x \text{ กับ training point } x^n$$

N คือ จำนวน training point ทั้งหมด

หากใช้วิธีวัดระยะทางแบบยุคลิด (Euclidean Distance) จะได้

$$d(x, x^n) = \sqrt{\sum_{i \in m} (x_i - x_i^n)^2} \quad \text{เมื่อ } m \text{ คือ จำนวนตัวแปรต้น (Independent Variable), จำนวนปัจจัย (Factor), หรือจำนวนคุณลักษณะ (Feature)}$$

ขั้นที่ 2 : หา training point x^n ซึ่งมีระยะทาง $d(x, x^n)$ ใกล้เคียงกับ test point x มากที่สุด จำนวน k อันดับแรก

ขั้นที่ 3 : กำหนดผลลัพธ์ (Output)

- สำหรับงาน Regression

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

เมื่อ $\hat{y}$ คือ ค่าที่ทำนายได้

y_i คือ ค่าที่เกิดขึ้นจริงของเพื่อนบ้านตัวที่ i
 $N_k(x)$ คือ เซตของเพื่อนบ้าน k อันดับแรก

- สำหรับงาน Classification

$$\hat{y} = \text{mode}(y_i \mid i \in N_k(x))$$

3.4.2 ตัวอย่างการคำนวณ KNN

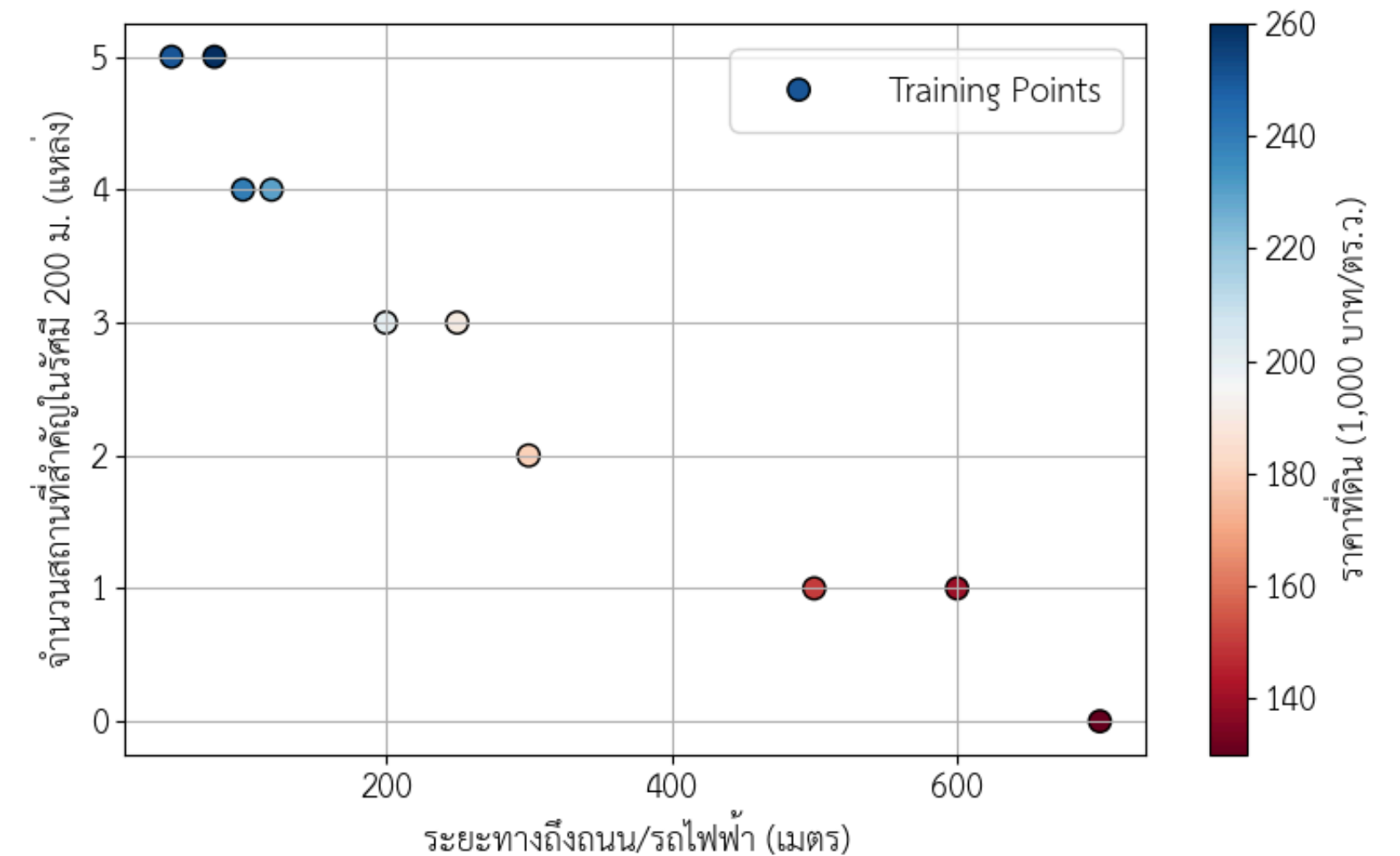
ข้อที่ 1 : จงแสดงวิธีการคำนวณ KNN เพื่อประเมินราคาที่ดิน

ตารางที่ 3-3ก : ชุดข้อมูลฝึก (Training Set)

รายการ	ระยะทางถึงถนน/ รถไฟฟ้า (เมตร)	จำนวนสถานที่สำคัญ ในรัศมี 200 ม. (แหล่ง)	ราคาที่ดิน (1,000 บาท/ตร.ว.)
1	50	5	250
2	100	4	240
3	80	5	260
4	200	3	200
5	500	1	150
6	300	2	180
7	700	0	130
8	600	1	140
9	250	3	190
10	120	4	230

ตารางที่ 3-3ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	ระยะทางถึงถนน/ รถไฟฟ้า (เมตร)	จำนวนสถานที่สำคัญ ในรัศมี 200 ม. (แหล่ง)	ราคาที่ดิน (1,000 บาท/ตร.ว.)
1	220	2	?



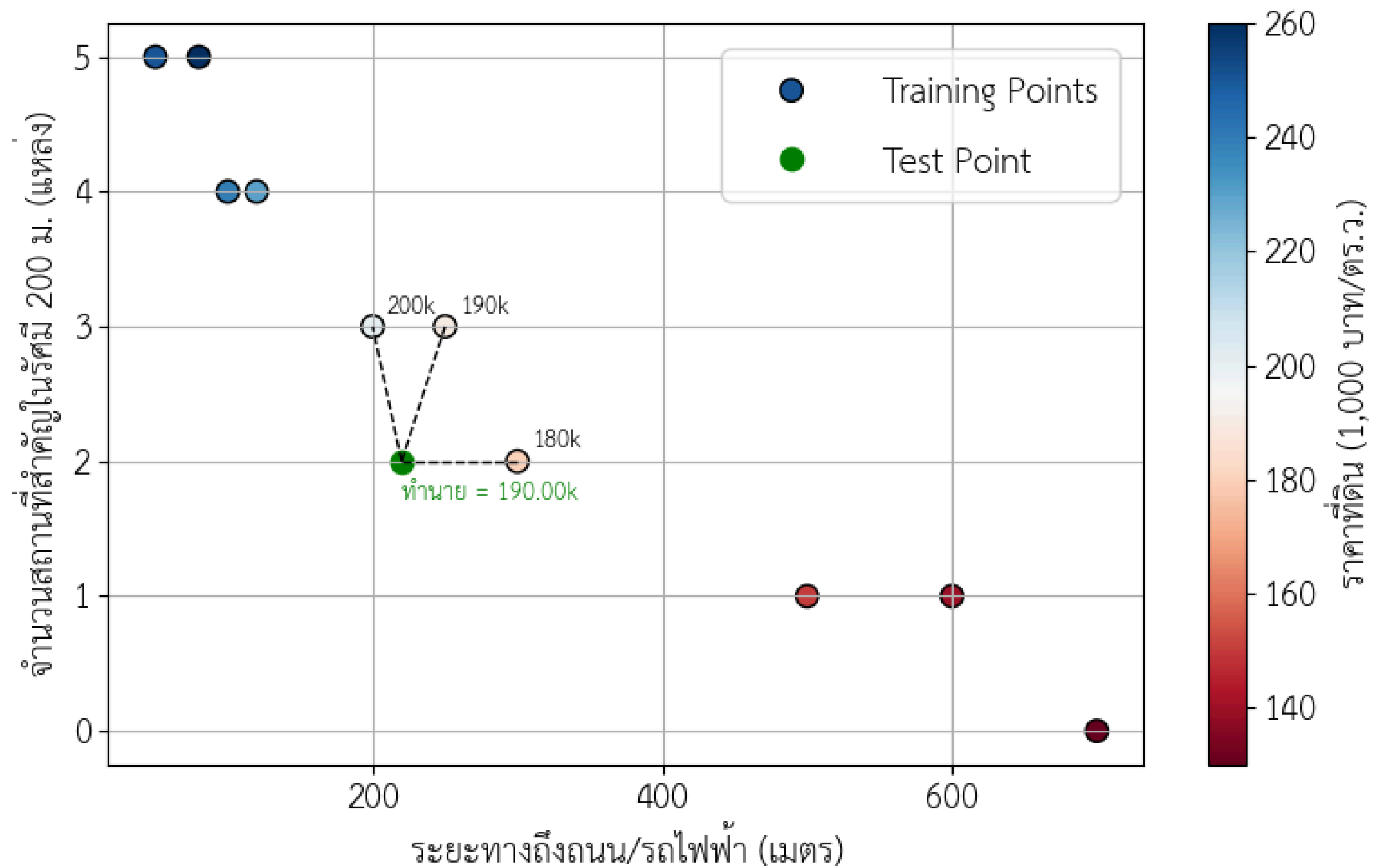
ภาพที่ 3-8 : ชุดข้อมูลฝึก (Training Set)

วิธีทำ

ชุดข้อมูลทดสอบ (Test Set)						
	ปัจจัยที่ 1 (x1)	ปัจจัยที่ 2 (x2)	ผลลัพธ์ (y)			
	220	2	190.....	(ค่าเฉลี่ยของคำตอบจากข้อมูลฝึก 3 อันดับแรก)		
ชุดข้อมูลฝึก (Traning Set)						
รายการที่	ปัจจัยที่ 1 (x1)	ปัจจัยที่ 2 (x2)	ผลลัพธ์ (y)	การคำนวณระยะทาง ด้วยวิธีแบบยุคลิด	ระยะทาง	รายการข้อมูลฝึก ที่ใกล้ที่สุด 3 อันดับ
1	50	5	250	$SQRT((220-50)^2 + (2-5)^2)$	170.03	
2	100	4	240	$SQRT((220-100)^2 + (2-4)^2)$	120.02	
3	80	5	260	$SQRT((220-80)^2 + (2-5)^2)$	140.03	
4	200	3	200	$SQRT((220-200)^2 + (2-3)^2)$	20.02	x
5	500	1	150	$SQRT((220-500)^2 + (2-1)^2)$	280	
6	300	2	180	$SQRT((220-300)^2 + (2-2)^2)$	80	x
7	700	0	130	$SQRT((220-700)^2 + (2-0)^2)$	480	
8	600	1	140	$SQRT((220-600)^2 + (2-1)^2)$	380	
9	250	3	190	$SQRT((220-250)^2 + (2-3)^2)$	30.02	x
10	120	4	230	$SQRT((220-120)^2 + (2-4)^2)$	100.02	

นำผลลัพธ์ 190 ไปคูณ 1,000 เพื่อปรับให้อยู่ในหน่วย บาท/ตร.ว. จะได้ $190 \times 1,000 = 190,000$

ตอบ ราคาที่ดินที่ประเมินได้ คือ 190,000 บาท/ตร.ว.



ภาพที่ 3-9 : การทดสอบ (Test) หรือการประเมินราคาที่ดิน

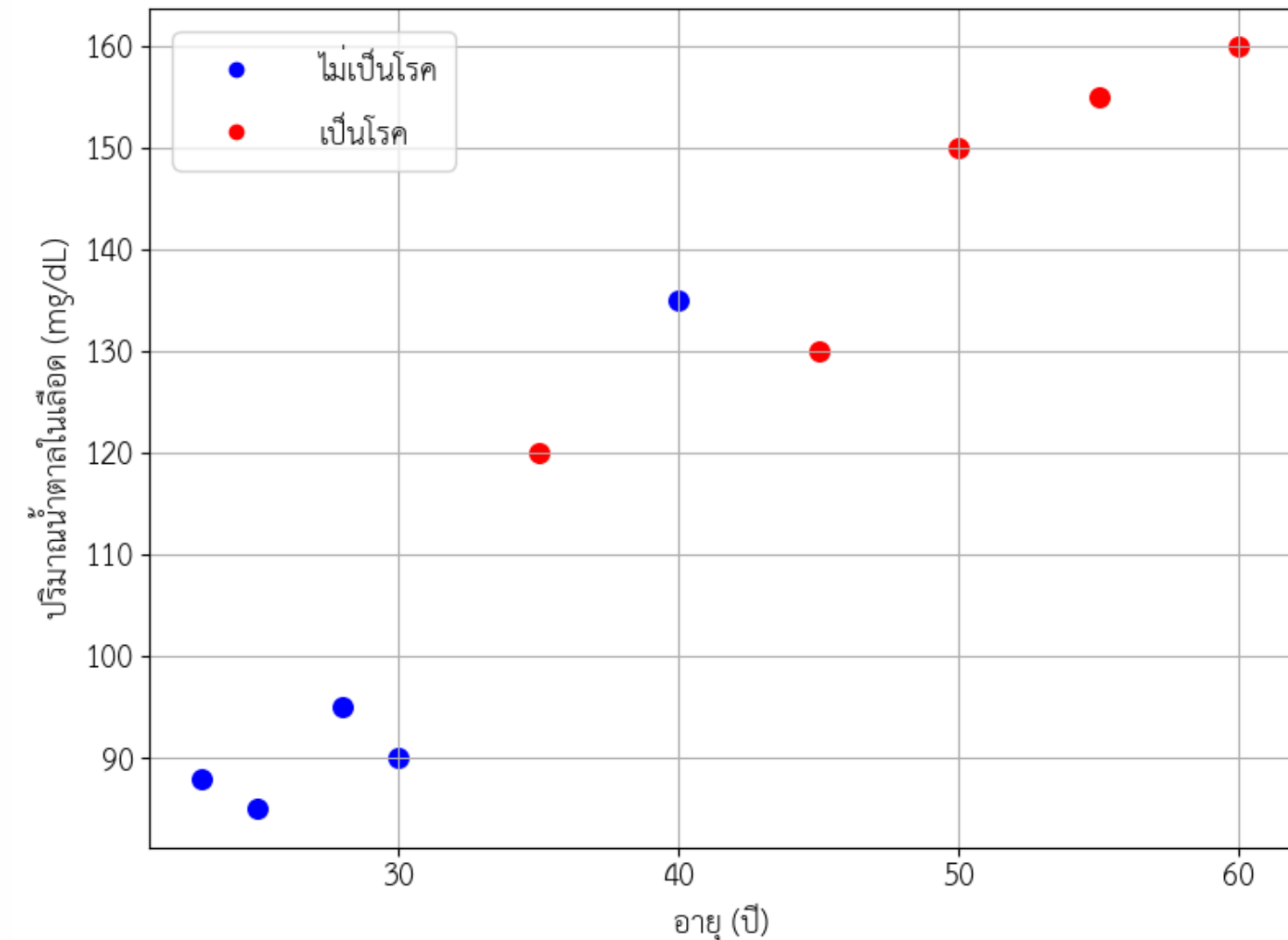
ข้อที่ 2 : แสดงวิธีการคำนวณ KNN เพื่อวินิจฉัยการป่วยเป็นโรคเบาหวาน

ตารางที่ 3-4ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	25	85	ไม่เป็นโรค
2	30	90	ไม่เป็นโรค
3	45	130	เป็นโรค
4	35	120	เป็นโรค
5	50	150	เป็นโรค
6	23	88	ไม่เป็นโรค
7	60	160	เป็นโรค
8	55	155	เป็นโรค
9	40	135	ไม่เป็นโรค
10	28	95	ไม่เป็นโรค

ตารางที่ 3-4ข : ชุดข้อมูลทดสอบ (Test Set)

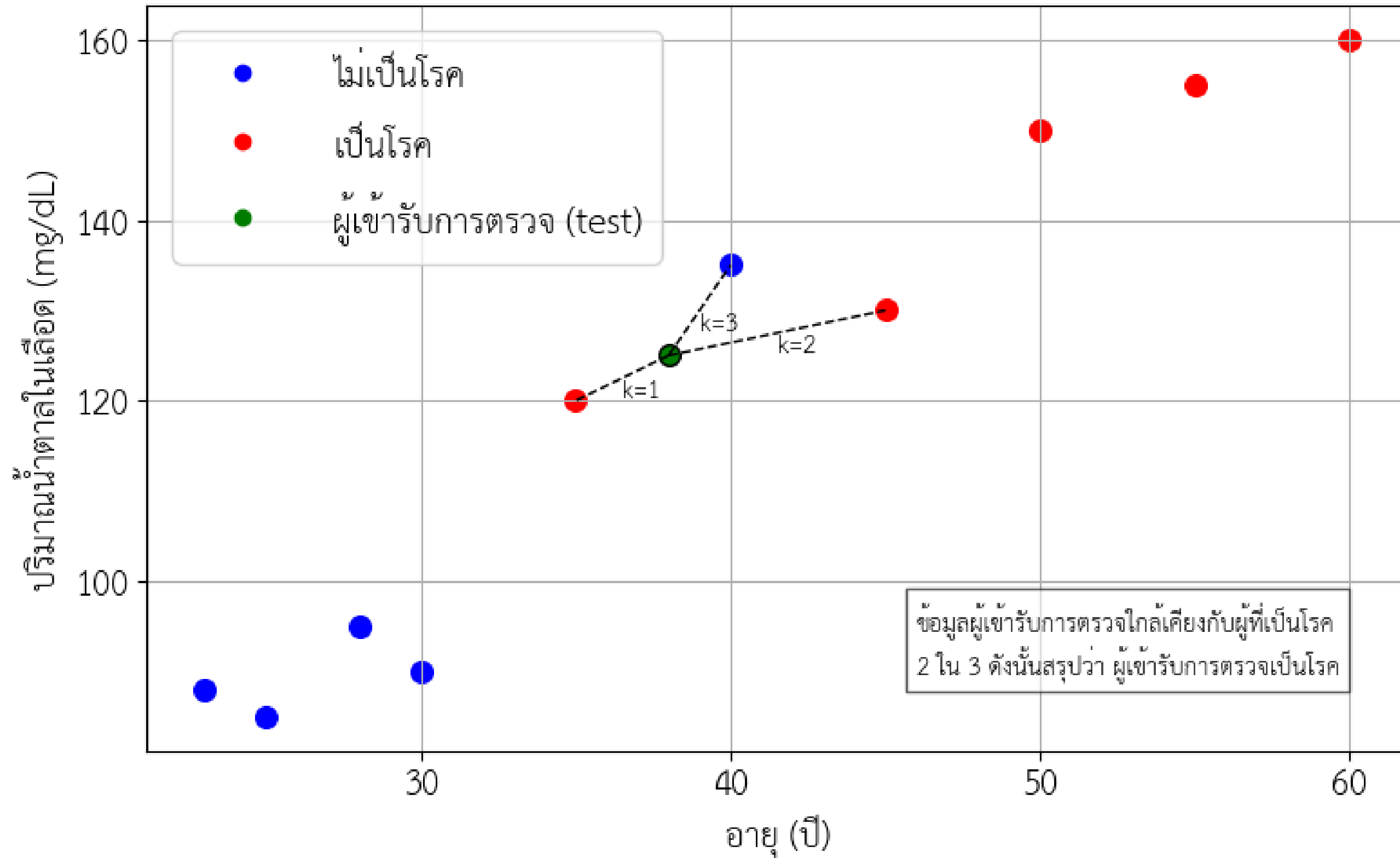
รายการ	อายุ (ปี)	ปริมาณน้ำตาลในเลือด (mg/dL)	การเป็นโรค
1	38	125	?



ภาพที่ 3-10 : ชุดข้อมูลฝึก (Training Set)

วิธีทำ

ชุดข้อมูลทดสอบ (Test Set)						
	ปัจจัยที่ 1 (x1)	ปัจจัยที่ 2 (x2)	ผลลัพธ์ (y)			
	38	125	เป็นโรค.....	(ค่าฐานนิยมของคำตอบจากข้อมูลฝึก 3 อันดับแรก)		
ชุดข้อมูลฝึก (Traning Set)						
รายการที่	ปัจจัยที่ 1 (x1)	ปัจจัยที่ 2 (x2)	ผลลัพธ์ (y)	การคำนวณระยะทาง ด้วยวิธีแบบยุคลิด	ระยะทาง	รายการข้อมูลฝึก ที่ใกล้เคียงที่สุด 3 อันดับ
1	25	85	ไม่เป็นโรค	$\text{SQRT}((38-25)^2 + (125-85)^2)$	42.06	
2	30	90	ไม่เป็นโรค	$\text{SQRT}((38-30)^2 + (125-90)^2)$	35.9	
3	45	130	เป็นโรค	$\text{SQRT}((38-45)^2 + (125-130)^2)$	8.6	x
4	35	120	เป็นโรค	$\text{SQRT}((38-35)^2 + (120-120)^2)$	5.83	x
5	50	150	เป็นโรค	$\text{SQRT}((38-50)^2 + (125-150)^2)$	27.73	
6	23	88	ไม่เป็นโรค	$\text{SQRT}((38-23)^2 + (125-88)^2)$	39.92	
7	60	160	เป็นโรค	$\text{SQRT}((38-60)^2 + (125-160)^2)$	41.34	
8	55	155	เป็นโรค	$\text{SQRT}((38-55)^2 + (125-155)^2)$	34.48	
9	40	135	ไม่เป็นโรค	$\text{SQRT}((38-40)^2 + (125-135)^2)$	10.2	x
10	28	95	ไม่เป็นโรค	$\text{SQRT}((38-28)^2 + (125-95)^2)$	31.62	



ภาพที่ 3-11 : การทดสอบ (Test) หรือการวินิจฉัย

9.2.3 ตัวอย่างโค้ด KNN

สำหรับข้อที่ 1

- เขียนโค้ดแบบไม่ใช้ไลบรารี ML

```
1  #Import Python library
2  import math
3  import numpy as np
4
5  #กำหนดค่าเริ่มต้น
6  x_train = [[50, 5],[100, 4], [80, 5], [200, 3], [500, 1], [300, 2], [700, 0], [600, 1], [250, 3],[120, 4]]
7  y_train = [250, 240, 260, 200, 150, 180, 130, 140, 190, 230]
8  x_test = [220, 2]
9  k = 3
10
11
12  #ขั้นที่ 1 : คำนวณระยะทางระหว่าง test point กับ training points
13  dist = []
14  for i in range(len(x_train)):
15      d = math.sqrt( pow(x_test[0] - x_train[i][0], 2) +
16                    pow(x_test[1] - x_train[i][1], 2) )
17      dist.append(round(d,2))
18
19  print('ขั้นที่ 1 : ระยะทาง คือ', dist)
20
```

```

21 #ขั้นที่ 2 : หา training points ซึ่งมีระยะทางใกล้เคียงกับ test point มากที่สุด จำนวน k อันดับแรก
22 sort_dist = sorted(dist.copy())[:k]
23
24 index = []
25 for s in sort_dist:
26     for i, value in enumerate(dist):
27         if s == value:
28             index.append(i)
29
30     if len(index) == k:
31         break;
32
33 print('ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ', [i + 1 for i in index])
34
35 #ขั้นที่ 3 : หาคำตอบจากตัวแปรตามของ traning points ที่ใกล้ที่สุด k ตัว
36 #หาค่า average สำหรับงาน regression
37 y_pred = [y_train[i] for i in index]
38 averaeg = np.mean(y_pred)*1000
39
40 print(f'ขั้นที่ 3   ราคาประเมินที่ดิน คือ {averaeg:,.2f} บาท/ตร.ว.')

```

ขั้นที่ 1 : ระยะทาง คือ [170.03, 120.02, 140.03, 20.02, 280.0, 80.0, 480.0, 380.0, 30.02, 100.02]

ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ [4, 9, 6]

ขั้นที่ 3 : ราคาประเมินที่ดิน คือ 190,000.00 บาท/ตร.ว.

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

```
1 #Import Python library
2 from sklearn.neighbors import KNeighborsRegressor
3
4 #Load Data
5 x_train = [[50, 5], [100, 4], [80, 5], [200, 3], [500, 1], [300, 2], [700, 0], [600, 1], [250, 3], [120, 4]]
6 y_train = [250, 240, 260, 200, 150, 180, 130, 140, 190, 230]
7 x_test = [220, 2]
8 k = 3
9
10 #Build Model
11 net = KNeighborsRegressor(n_neighbors = k, metric = 'minkowski', p = 2)
12 net.fit(x_train, y_train)
13
14 #Test Model
15 y_pred = net.predict([x_test])*1000
16 print(f'ราคาประเมินที่ดิน คือ {y_pred[0]:.2f} บาท/ตร.ว.')
```

ราคาประเมินที่ดิน คือ 190,000.00 บาท/ตร.ว.

สำหรับข้อที่ 2

- เขียนโค้ดแบบไม่ใช้ไลบรารี ML

```
1 #Import Python library
2 import math
3 from scipy import stats
4
5 #กำหนดค่าเริ่มต้น
6 x_train = [[25, 85], [30, 90], [45, 130], [35, 120], [50, 150], [23, 88], [60, 160], [55, 155], [40, 135], [28, 95]]
7 y_train = [0, 0, 1, 1, 1, 0, 1, 1, 0, 0]
8 x_test = [38, 125]
9 k = 3
10
11
12 #ขั้นที่ 1 . คำนวณระยะทางระหว่าง test point กับ training points
13 dist = []
14 for i in range(len(x_train)):
15     d = math.sqrt( pow(x_test[0] - x_train[i][0], 2) +
16                   pow(x_test[1] - x_train[i][1], 2) )
17     dist.append(round(d,2))
18
19 print('ขั้นที่ 1   ระยะทาง คือ', dist)
20
```

```

21 #ขั้นที่ 2 : หา training points ซึ่งมีระยะทางใกล้เคียงกับ test point มากที่สุด จำนวน k อันดับแรก
22 sort_dist = sorted(dist.copy())[:k]
23
24 index = []
25 for s in sort_dist:
26     for i, value in enumerate(dist):
27         if s == value:
28             index.append(i)
29
30     if len(index) == k:
31         break;
32
33 print('ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ', [i + 1 for i in index])
34
35 #ขั้นที่ 3 : หาคำตอบจากตัวแปรตามของ training points ที่ใกล้ที่สุด k ตัว
36 #หาค่า mode สำหรับงาน classification
37 y_pred = [y_train[i] for i in index]
38 mode = int(stats.mode(y_pred)[0])
39
40 print('ขั้นที่ 3 : ผลการวินิจฉัย คือ', "เป็นโรค" if mode else "ไม่เป็นโรค")

```

ขั้นที่ 1 : ระยะทาง คือ [42.06, 35.9, 8.6, 5.83, 27.73, 39.92, 41.34, 34.48, 10.2, 31.62]
 ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ [4, 3, 9]
 ขั้นที่ 3 : ผลการวินิจฉัย คือ เป็นโรค

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

```
1 #Import Python library
2 import pandas as pd
3 from sklearn.neighbors import KNeighborsClassifier
4
5 #Load Data
6 x = pd.read_csv('data/diabetes.csv')
7 x_train = x[['age','blood_glucose']].to_numpy()
8 y_train = x['diabetes']
9 x_test = [38, 125]
10 k = 3
11
12 #Build and Train Model
13 net = KNeighborsClassifier(n_neighbors = k, metric = 'minkowski', p = 2)
14 net.fit(x_train,y_train)
15
16 #Test Model
17 y_pred=net.predict([x_test])
18 print('ผลการวินิจฉัย คือ ' y_pred[0])
```

ผลการวินิจฉัย คือ เป็นโรค

3.5 การตรวจสอบโมเดล (Model Validation)

การตรวจสอบโมเดล (Model Validation) เป็นวิธีการที่ช่วยให้การประเมินประสิทธิภาพของโมเดล (Performance Evaluation) มีความน่าเชื่อถือมากขึ้น ประกอบด้วยเทคนิคต่าง ๆ ดังนี้

- Hold Out
- Repeat Random Sub-sampling
- Cross-Validation

3.5.1 Hold Out

- แบ่งข้อมูล Training/Test data เพียงครั้งเดียว
- เป็นวิธีที่สะดวก
- ประสิทธิภาพ/ประสิทธิผล มีความแปรปรวนสูง
- (การสุ่มอาจจะทำให้ได้ Training set และ Test set แต่ละครั้งต่างกัน)
- มีแค่บาง samples ถูกใช้ในการ Test
- ตัวอย่างการแบ่งแบบ Hold out เช่น
 - แบ่ง 70/30 (Training Data=70%, Test Data=30%)
 - แบ่ง 80/20 (Training Data=80%, Test Data=20%)

โค้ดไพทอน สำหรับ Hold Out

```
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3 x = np.array(
4     [[1, 2],
5      [8, 4],
6      [9, 6],
7      [7, 8],
8      [4, 3]])
9 y = np.array([1, 7, 8, 4, 2])
10 x_train, x_test, y_train, y_test = train_test_split(x, y,
11     |         |         |         |         |         |         |         |
12     test_size=0.3, random_state=0)
13
14 print("x_train:\n", x_train)
15 print("\nx_test:\n", x_test)
16 print("\ny_train:\n", y_train)
17 print("\ny_test:\n", y_test)
```

x_train:	x_test:	y_train:	y_test:
[[8 4]	[[9 6]	[7 4 2]	[8 1]
[7 8]	[1 2]]		
[4 3]]			

Method: train_test_split

Parameter:

x คือ ข้อมูลตัวแปรต้นทุก records

y คือ ข้อมูลตัวแปรตามทุก records

test_size=0.3 คือ การสุ่มเอา Test data 30% ของทั้งหมด

random_state=0 คือ การสุ่มลำดับเดิมเสมอ ค่าอยู่ในช่วง [0 - 2³²]

shuffle=False คือ ไม่มีการสุ่ม ให้ใช้ลำดับเดิม

Output:

x_train คือ Training data ส่วนที่เป็นตัวแปรต้น

x_test คือ Test data ส่วนที่เป็นตัวแปรต้น

y_train คือ Training data ส่วนที่เป็นตัวแปรตาม

y_test คือ Test data ส่วนที่เป็นตัวแปรตาม

3.5.2 Repeat Random Sub-sampling

- เป็นวิธีการที่ปรับปรุงจาก Hold out
- สุ่มแบ่งข้อมูล Training/Test data หลายครั้ง แทนการแบ่งแค่ครั้งเดียว
- ประสิทธิภาพ/ประสิทธิผล ประปรวนลดลงจาก Hold out
- ตัวอย่างการแบ่งแบบ Repeat random sub-sampling เช่น
 - แบ่ง 70/30 จำนวน 20 รอบ
 - แบ่ง 80/20 จำนวน 20 รอบ

โค้ดไพทอน สำหรับ Repeat Random Sub-sampling

```
1  import numpy as np
2  from sklearn.model_selection import ShuffleSplit
3  x = np.array(
4      [[1, 2],
5       [8, 4],
6       [9, 6],
7       [7, 8],
8       [4, 3]])
9  y = np.array([1, 7, 8, 4, 2])
10 ss = ShuffleSplit(n_splits=20,
11                  test_size=0.3, random_state=0).split(x)
12 for i, (train_index, test_index) in enumerate(ss):
13     print(f"Round {i+1}:")
14     print(f"    Train index={train_index}")
15     print(f"    Test index={test_index}")
16     x_train, x_test = x[train_index], x[test_index]
17     y_train, y_test = y[train_index], y[test_index]
```

Method: ShuffleSplit

Parameter:

n_splits=20 คือ การสุ่มเอาข้อมูล จำนวน 20 ชุด
test_size=0.3 คือ การสุ่มเอา Test data 30% ของทั้งหมด
random_state=0 คือ เป็นการสุ่มลำดับเดิมเสมอ
ถ้าต้องการให้ สุ่มใหม่ทุกครั้งให้กำหนดเป็น "None"
x คือ ข้อมูลตัวแปรต้นทุก records

Output:

train_index คือ Index ของ Training data

test_index คือ Index ของ Test data

Round 1:

Train index=[1 3 4]

Test index=[2 0]

Round 2:

Train index=[1 4 3]

Test index=[0 2]

⋮

Round 20:

Train index=[1 2 0]

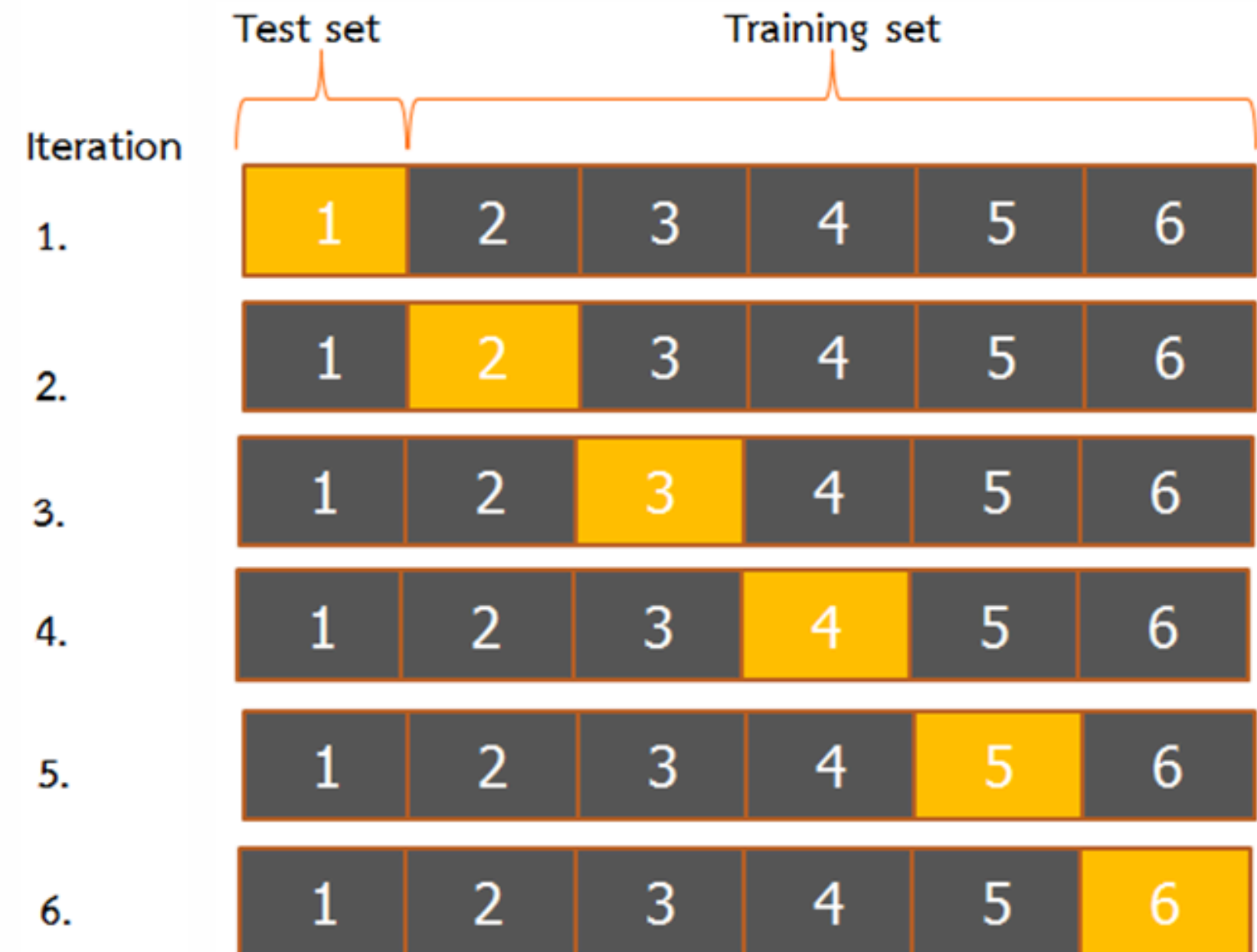
Test index=[3 4]

3.5.3 Cross-Validation

- เป็นวิธีการที่ปรับปรุงจาก Hold out และ Repeat random sub-sampling
- แบ่งข้อมูลครั้งเดียวแต่แบ่งออกเป็นหลายกอง (Fold) แต่ละกองจะมีข้อมูลไม่ซ้ำกับข้อมูลกองอื่น
- ประสิทธิภาพ/ประสิทธิผล ประปรวนลดลงจาก Repeat random sub-sampling
- ตัวอย่างการแบ่งแบบ Cross-validation เช่น
 - k -Fold Cross Validation
 - 3-Fold Cross Validation : แบ่งข้อมูลออกเป็น 3 กอง
 - 6-Fold Cross Validation : แบ่งข้อมูลออกเป็น 6 กอง
 - 10-Fold Cross Validation : แบ่งข้อมูลออกเป็น 10 กอง
 - Stratified k -Fold Cross Validation : จัดสรรให้ข้อมูลแต่ละกองมีจำนวน Class เท่า ๆ กัน
 - n -Fold Cross Validation หรือ Leave-One-Out Cross Validation (LOOCV) : แบ่งข้อมูลออกเป็น n กอง กองละ 1 record

ขั้นตอนการแบ่งข้อมูลด้วย k -Fold Cross Validation

- สุ่มแยกข้อมูลจาก dataset ออกเป็น k กอง (ในที่นี้กำหนด $k=6$)
- รอบแรกใช้กองที่ 1 เป็นข้อมูลสำหรับ Test set แล้วนำ 5 กองที่เหลือมารวมกันเพื่อใช้เป็น Training set
- รอบสองใช้กองที่ 2 เป็น test set แล้วใช้ 5 กองที่เหลือรวมเป็น Training set
- ทำเช่นนี้ไปจนครบ 6 รอบ
- ในแต่ละรอบจะใช้ข้อมูล Test set เพื่อวัดประสิทธิภาพของโมเดล ด้วย Evaluation Metrics เช่น MAPE, Accuracy, Precision, Recall แล้วหาค่าเฉลี่ยจากทั้ง 6 รอบ เพื่อกำหนดเป็นประสิทธิภาพของโมเดล



ภาพที่ 3-12 : 6-fold cross validation

โค้ดไพทอน สำหรับ *k*-fold Cross Validation

```
1  import numpy as np
2  from sklearn.model_selection import KFold
3  x = np.array(
4      [[1, 2],
5       [8, 4],
6       [9, 6],
7       [7, 8],
8       [4, 3]])
9  y = np.array([1, 7, 8, 4, 2])
10 kf = KFold(n_splits=3, shuffle=True, random_state=0).split(x)
11 for i, (train_index, test_index) in enumerate(kf):
12     print(f"Fold {i+1}:")
13     print(f"  Train index={train_index}")
14     print(f"  Test index={test_index}")
15     x_train, x_test = x[train_index], x[test_index]
16     y_train, y_test = y[train_index], y[test_index]
```

Method: KFold

Parameter:

n_splits=3 คือ จำนวนกองของข้อมูลที่ต้องการแบ่ง

shuffle=True คือ กำหนดให้มีการสุ่มข้อมูล

random_state=0 คือ เป็นการสุ่มลำดับเดิมเสมอ

x คือ ข้อมูลตัวแปรต้นทุก records

Output:

train_index คือ Index ของ Training data

test_index คือ Index ของ Test data

Fold 1:

Train index=[1 3 4]

Test index=[0 2]

Fold 2:

Train index=[0 2 4]

Test index=[1 3]

Fold 3:

Train index=[0 1 2 3]

Test index=[4]

โค้ดไพทอน สำหรับ Stratified k -Fold Cross Validation

```
1 import numpy as np
2 from sklearn.model_selection import StratifiedKFold
3 x = np.array(
4     [[1, 2],
5      [8, 4],
6      [9, 6],
7      [7, 8],
8      [4, 3]])
9 y = np.array([1, 1, 0, 0, 1])
10 ss = StratifiedKFold(n_splits=3, shuffle=True, random_state=0).split(x, y)
11 for i, (train_index, test_index) in enumerate(ss):
12     print(f"Fold {i+1}:")
13     print(f"  Train index={train_index}")
14     print(f"  Test index={test_index}")
15     x_train, x_test = x[train_index], x[test_index]
16     y_train, y_test = y[train_index], y[test_index]
```

Method: StratifiedKFold

Parameter:

n_splits=3 คือ จำนวนกองของข้อมูลที่ต้องการแบ่ง

shuffle=True คือ กำหนดให้มีการสุ่มข้อมูล

random_state=0 คือ เป็นการสุ่มลำดับเดิมเสมอ

x คือ ข้อมูลตัวแปรต้นทุก records

y คือ ข้อมูลตัวแปรตามทุก records

Output:

train_index คือ Index ของ Training data

test_index คือ Index ของ Test data

Fold 1:

Train index=[0 1 3]

Test index=[2 4]

Fold 2:

Train index=[0 2 4]

Test index=[1 3]

Fold 3:

Train index=[1 2 3 4]

Test index=[0]

โค้ดไพทอน สำหรับ LOOCV

```
1  import numpy as np
2  from sklearn.model_selection import LeaveOneOut
3  x = np.array(
4      [[1, 2],
5       [8, 4],
6       [9, 6],
7       [7, 8],
8       [4, 3]])
9  y = np.array([1, 7, 8, 4, 2])
10 loo = LeaveOneOut().split(x)
11 for i, (train_index, test_index) in enumerate(loo):
12     print(f"Fold {i+1}:")
13     print(f"    Train index={train_index}")
14     print(f"    Test index={test_index}")
15     x_train, x_test = x[train_index], x[test_index]
16     y_train, y_test = y[train_index], y[test_index]
```

Method: LeaveOneOut

Parameter:

x คือ ข้อมูลตัวแปรต้นทุก records

Output:

train_index คือ Index ของ Training data

test_index คือ Index ของ Test data

Fold 1:

Train index=[1 2 3 4]

Test index=[0]

Fold 2:

Train index=[0 2 3 4]

Test index=[1]

Fold 3:

Train index=[0 1 3 4]

Test index=[2]

Fold 4:

Train index=[0 1 2 4]

Test index=[3]

Fold 5:

Train index=[0 1 2 3]

Test index=[4]

3.6 การประเมินประสิทธิภาพโมเดล (Performance Evaluation)

การประเมินประสิทธิภาพโมเดล (Performance Evaluation) คือกระบวนการวัดผลการทำงานของโมเดลโดยใช้เมตริกการประเมิน (Evaluation Metrics) เพื่อประเมินประสิทธิภาพของโมเดลในการแก้ปัญหา

- Evaluation Metrics (Regression)
- Evaluation Metrics (Classification)

3.6.1 Evaluation Metrics (Regression)

- **Evaluation Metrics**

- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)
- Mean Square Error (MSE)
- Root Mean Square Error (RMSE)
- R-squared (R^2)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \frac{|\hat{y}_i - y_i|}{y_i} \times 100$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

เมื่อ

$\hat{y}_i$ คือ ค่าที่ทำนายได้ ของข้อมูลแถว (record) ที่ i

y_i คือ ค่าที่เกิดขึ้นจริง ของข้อมูลแถว (record) ที่ i

n คือ จำนวนข้อมูลทดสอบ (Test Points) ทั้งหมด

- ตัวอย่างการคำนวณ สำหรับ Performance Evaluation (Regression)

จงคำนวณค่า MAE, MAPE, MSE, RMSE, R^2 เมื่อค่าที่เกิดขึ้นจริง (y_{test}) และค่าที่ทำนายได้ (y_{pred}) เป็นดังนี้

$y_{\text{test}} = [1, 7, 8, 4, 2]$ #ค่าที่เกิดขึ้นจริง

$y_{\text{pred}} = [2, 6, 10, 3, 2]$ #ค่าที่ทำนายได้

วิธีทำ

$$\begin{aligned}\text{MAE} &= \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \\ &= \frac{1}{5} (|2 - 1| + |6 - 7| + |10 - 8| + |3 - 4| + |2 - 2|) = \frac{1}{5} (5) = 1\end{aligned}$$

$$\begin{aligned}\text{MAPE} &= \frac{1}{n} \sum_{i=1}^n \frac{|\hat{y}_i - y_i|}{y_i} \times 100 \\ &= \frac{1}{5} \left(\frac{|2 - 1|}{1} + \frac{|6 - 7|}{7} + \frac{|10 - 8|}{8} + \frac{|3 - 4|}{4} + \frac{|2 - 2|}{2} \right) \times 100 \\ &= \frac{1}{5} (1.6429) \times 100 = 32.86\%\end{aligned}$$

$$\begin{aligned}\text{MSE} &= \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \\ &= \frac{1}{5} ((2 - 1)^2 + (6 - 7)^2 + (10 - 8)^2 + (3 - 4)^2 + (2 - 2)^2) = \frac{1}{5} (7) = 1.4\end{aligned}$$

$$\begin{aligned}\text{RMSE} &= \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2} \\ &= \sqrt{1.4} = 1.18\end{aligned}$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

y_test = [1, 7, 8, 4, 2] #ค่าที่เกิดขึ้นจริง
y_pred = [2, 6, 10, 3, 2] #ค่าที่ทำนายได้

- คำนวณแต่ละส่วนของ R^2

$$\sum_{i=1}^n (\hat{y}_i - y_i)^2 = (2 - 1)^2 + (6 - 7)^2 + (10 - 8)^2 + (3 - 4)^2 + (2 - 2)^2 = 7$$

$$\bar{y} = \frac{1}{5}(1 + 7 + 8 + 4 + 2) = 4.4$$

$$\sum_{i=1}^n (y_i - \bar{y})^2 = (1 - 4.4)^2 + (7 - 4.4)^2 + (8 - 4.4)^2 + (4 - 4.4)^2 + (2 - 4.4)^2 = 37.2$$

- คำนวณรวม

$$R^2 = 1 - \frac{7}{37.2} = 0.81$$

โค้ดไพทอน สำหรับ Performance Evaluation (Regression)

```
1 from sklearn.metrics import mean_absolute_error
2 from sklearn.metrics import mean_absolute_percentage_error
3 from sklearn.metrics import mean_squared_error
4 from sklearn.metrics import root_mean_squared_error
5 from sklearn.metrics import r2_score
6
7 y_test=[1, 7, 8, 4, 2] #ค่าที่เกิดขึ้นจริง
8 y_pred=[2, 6, 10, 3, 2] #ค่าที่ทำนายได้จากโมเดล
9
10 mae=mean_absolute_error(y_test , y_pred)
11 mape=mean_absolute_percentage_error(y_test, y_pred)*100
12 mse=mean_squared_error(y_test, y_pred)
13 rmse=root_mean_squared_error(y_test, y_pred)
14 r2 = r2_score(y_test, y_pred)
15
16 print('MAE :', round(mae,2))
17 print('MAPE :', round(mape,2))
18 print('MSE :', round(mse,2))
19 print('RMSE :', round(rmse,2))
20 print('R-squared :', round(r2,2))
```

MAE : 1.0

MAPE : 32.86

MSE : 1.4

RMSE : 1.18

R-squared : 0.81

3.6.2 Evaluation Metrics (Classification)

- **Confusion Matrix**

		Predicted Value	
		0	1
Actual Value	0	TN	FP
	1	FN	TP

TP = True Positive (ทำนายว่าจริง ค่าเป็นจริง)

TN = True Negative (ทำนายว่าเท็จ ค่าเป็นเท็จ)

FP = False Positive (ทำนายว่าจริง ค่าเป็นเท็จ)

FN = False Negative (ทำนายว่าเท็จ ค่าเป็นจริง)

- **Evaluation Metrics**

- $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$
- $\text{Precision} = TP / (TP + FP)$
- $\text{Recall/Sensitivity} = TP / (TP + FN)$
- $\text{Specificity} = TN / (TN + FP)$
- $\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

- ตัวอย่างการคำนวณ สำหรับ Performance Evaluation (Classification)

จงคำนวณหาค่า Accuracy, Precision, Recall, Specificity, และ F1-score เมื่อค่าที่เกิดขึ้นจริง (y_test) และค่าที่ทำนายได้ (y_pred) เป็นดังนี้

y_test = [1, 1, 0, 0, 1] #ค่าที่เกิดขึ้นจริง

y_pred = [1, 1, 0, 1, 1] #ค่าที่ทำนายได้

วิธีทำ

- คำนวณ Confusion Matrix

ลำดับข้อมูล	ค่าที่ทำนายได้	ค่าที่เกิดขึ้นจริง	TP	TN	FP	FN
1	1	1	1			
2	1	1	1			
3	0	0		1		
4	1	0			1	
5	1	1	1			
รวม			3	1	1	0

		Predicted Value	
		0	1
Actual Value	0	1	1
	1	0	3

		Predicted Value	
		0	1
Actual Value	0	TN	FP
	1	FN	TP

- คำนวณ Evaluation Metrics

Accuracy = $(TP + TN) / (TP + TN + FP + FN) = (3 + 1) / (3 + 1 + 1 + 0) = 4 / 5 = 0.8$

Precision = $TP / (TP + FP) = 3 / (3 + 1) = 3 / 4 = 0.75$

Recall/Sensitivity = $TP / (TP + FN) = 3 / (3 + 0) = 3 / 3 = 1.00$

Specificity = $TN / (TN + FP) = 1 / (1 + 1) = 1 / 2 = 0.5$

F1-score = $2 \times (Precision \times Recall) / (Precision + Recall) = 2 \times (0.75 \times 1) / (0.75 + 1) = 0.86$

- โค้ดไพทอน สำหรับ Performance Evaluation (Classification)

```

1  from sklearn.metrics import confusion_matrix
2  from sklearn.metrics import classification_report
3
4  y_test=[1, 1, 0, 0, 1] #ค่าที่เกิดขึ้นจริง
5  y_pred=[1, 1, 0, 1, 1] #ค่าที่ทำนายได้จากโมเดล
6
7  print('Confustion matrix: ')
8  print(confusion_matrix(y_test, y_pred))
9
10 print('\nEvaluation Metrics: ')
11 print(classification_report(y_test,y_pred))

```

		Predicted Value	
		0	1
Actual Value	0	TN	FP
	1	FN	TP

Confustion matrix:

```
[[1 1]
 [0 3]]
```

Evaluation Metrics:

	precision	recall	f1-score	support
0	1.00	0.50	0.67	2
1	0.75	1.00	0.86	3
accuracy			0.80	5
macro avg	0.88	0.75	0.76	5
weighted avg	0.85	0.80	0.78	5

3.7 แบบฝึกหัด (Exercise)

ตารางที่ 3-5 : ชุดข้อมูลราคาอสังหาริมทรัพย์ (สำหรับข้อ 1-2)

ตารางที่ 3-5ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ (ปี)	ราคาอสังหาริมทรัพย์ (1,000 บาท)
1	20	25
2	25	30
3	29	45
4	32	70
5	35	100
6	45	300
7	50	500
8	55	693
9	60	783
10	65	863
11	72	940
12	80	950

ตารางที่ 3-5ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	ราคาอสังหาริมทรัพย์ (1,000 บาท)
1	30	?
2	70	?

1. จงแสดงวิธีการคำนวณ OLS เพื่อประเมินราคาอสังหาริมทรัพย์

2. จงวาดกราฟความสัมพันธ์ระหว่างอายุและราคาอสังหาริมทรัพย์ แล้ววิเคราะห์ว่าควรใช้เทคนิคการเรียนรู้ของเครื่องไหนแทน OLS เพราะอะไร

ตารางที่ 3-6 : ชุดข้อมูลอัตราการลาออกจากงาน (สำหรับข้อ 3-4)

ตารางที่ 3-6ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ	เพศ	ระดับการศึกษา	ลาออก
1	21-30	หญิง	ปริญญาโท	ใช่
2	21-30	ชาย	ปริญญาโท	ไม่ใช่
3	31-40	ชาย	ปริญญาตรี	ใช่
4	21-30	หญิง	ปริญญาเอก	ไม่ใช่
5	41-50	หญิง	ปริญญาโท	ใช่
6	31-40	หญิง	ปริญญาโท	ไม่ใช่
7	21-30	หญิง	ปริญญาโท	ไม่ใช่
8	41-50	หญิง	ปริญญาเอก	ไม่ใช่
9	21-30	ชาย	ปริญญาตรี	ไม่ใช่
10	31-40	ชาย	ปริญญาเอก	ใช่

ตารางที่ 3-6ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ	เพศ	ระดับการศึกษา	ลาออก
1	21-30	ชาย	ปริญญาตรี	?

3. จงแสดงวิธีการคำนวณ LR เพื่อวิเคราะห์การลาออกจากงาน

4. จงแสดงวิธีการคำนวณ KNN เพื่อวิเคราะห์การลาออกจากงาน กำหนดให้ $k=3$

ตารางที่ 3-7 : ชุดข้อมูลราคาบ้าน (สำหรับข้อ 5)

ตารางที่ 3-7ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุบ้าน (ปี)	ระยะทางไปรถไฟฟ้า (กิโลเมตร)	จำนวนร้านสะดวกซื้อ (ร้าน)	ราคาบ้าน (1,000 บาท/ตร.ม.)
1	7	0.4	3	139
2	4	0.2	7	159
3	3	0.6	4	60
4	10	1.2	2	25
5	8	1.4	1	26
6	3	0.5	5	65
7	2	0.2	3	100
8	3	0.1	7	160
9	2	0.3	2	134
10	9	0.9	1	40

ตารางที่ 3-7ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุบ้าน (ปี)	ระยะทางไปรถไฟฟ้า (กิโลเมตร)	จำนวนร้านสะดวกซื้อ (ร้าน)	ราคาบ้าน (1,000 บาท/ตร.ม.)
1	5	0.7	4	?

5. จงแสดงวิธีการคำนวณ KNN เพื่อประเมินราคาบ้าน **กำหนดให้** $k=3$



THANK → YOU!

Get In Touch With Us!

Phone

0-2692-2360-4

Facebook

<https://www.facebook.com/ComSci.CPC/>

Address

58 ถนนวิภาวดีรังสิต แขวง
รัชดาภิเษก เขตดินแดง

